

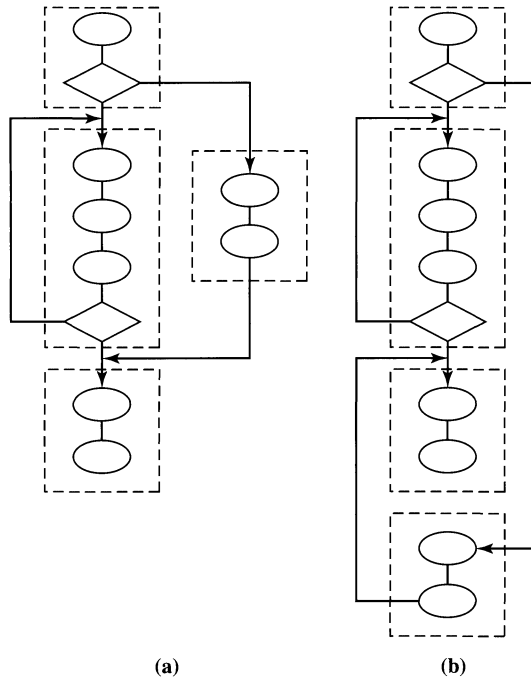


Figure 5.1

Program Control Flow: (a) The Control Flow Graph (CFG);
(b) Mapping the CFG to Sequential Memory Locations.

5.1.2 Performance Degradation Due to Branches

A pipelined machine achieves its maximum throughput when it is in the streaming mode. For the fetch stage, streaming mode implies the continuous fetching of instructions from sequential locations in the program memory. Whenever the control flow of the program deviates from the sequential path, potential disruption to the streaming mode can occur. For unconditional branches, subsequent instructions cannot be fetched until the target address of the branch is determined. For conditional branches, the machine must wait for the resolution of the branch condition, and if the branch is to be taken, it must further wait until the target address is available. Figure 5.2 illustrates the disruption of the streaming mode by branch instructions. Branch instructions are executed by the branch functional unit. For a conditional branch, it is not until it exits the branch unit and when both the branch condition and the branch target address are known that the fetch stage can correctly fetch the next instruction.

As Figure 5.2 illustrates, this delay in processing conditional branches incurs a penalty of three cycles in fetching the next instruction, corresponding to the traversal of the decode, dispatch, and execute stages by the conditional branch. The actual lost-opportunity cost of three stalled cycles is not just three empty instruction

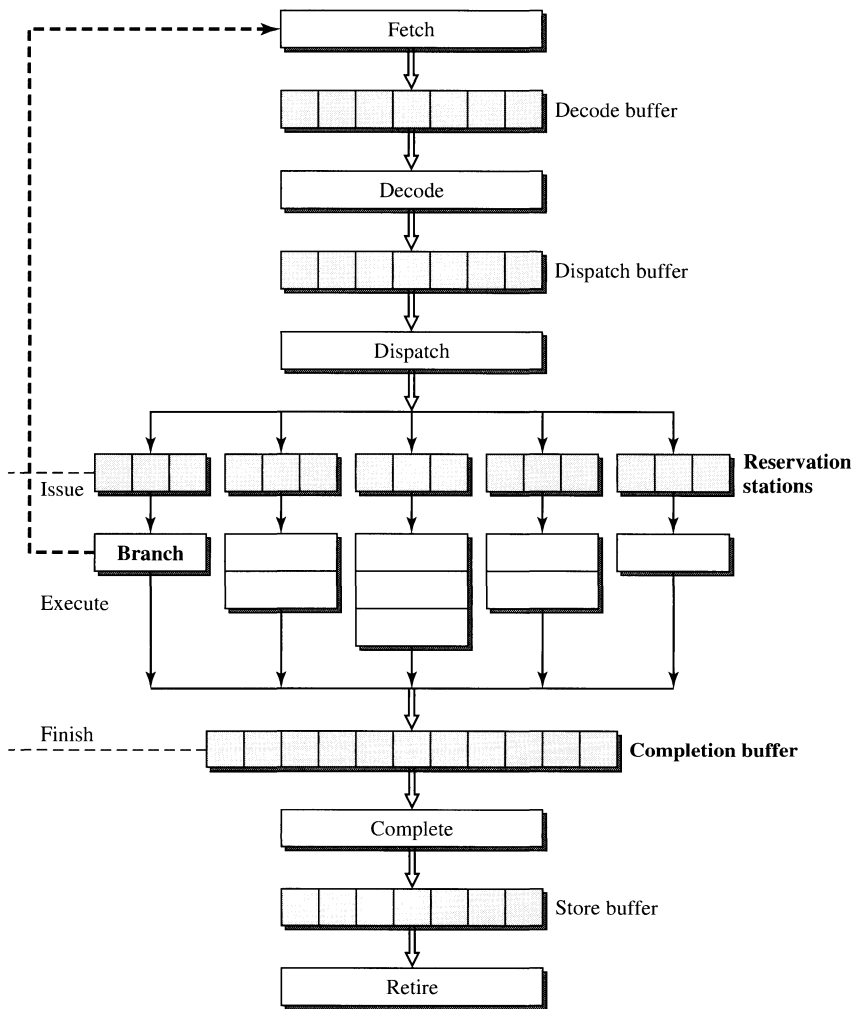
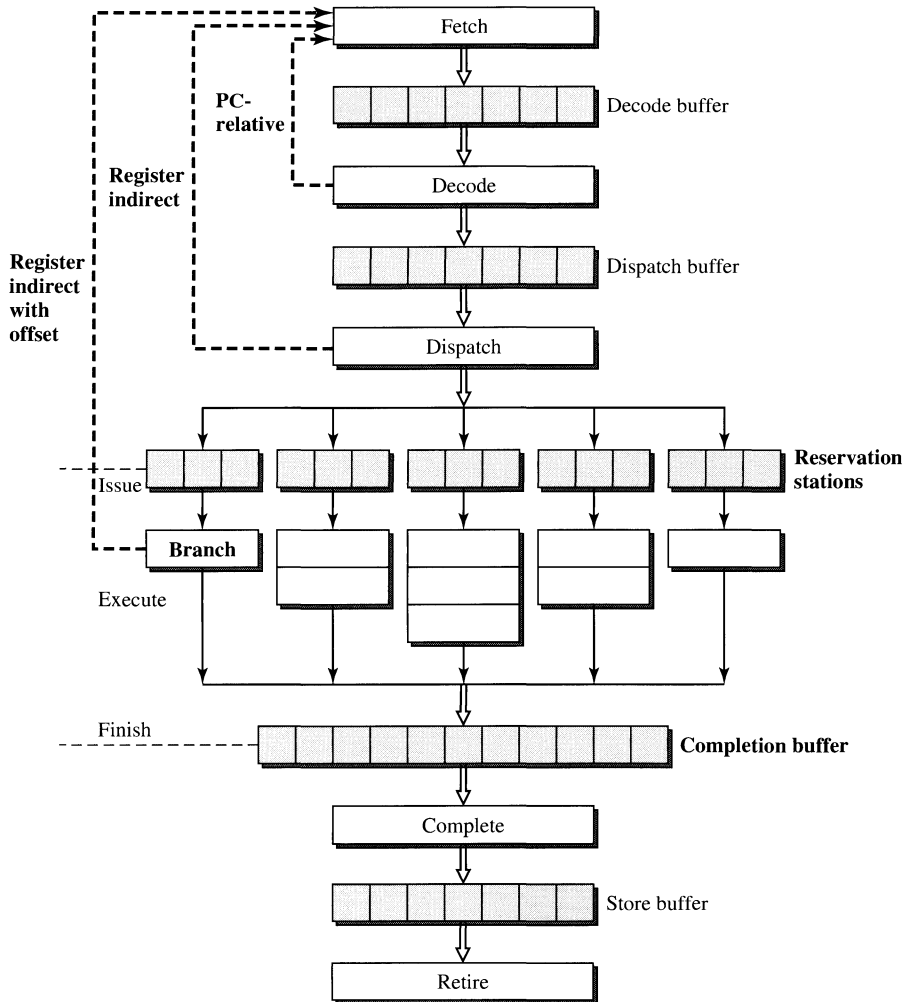


Figure 5.2
Disruption of Sequential Control Flow by Branch Instructions.

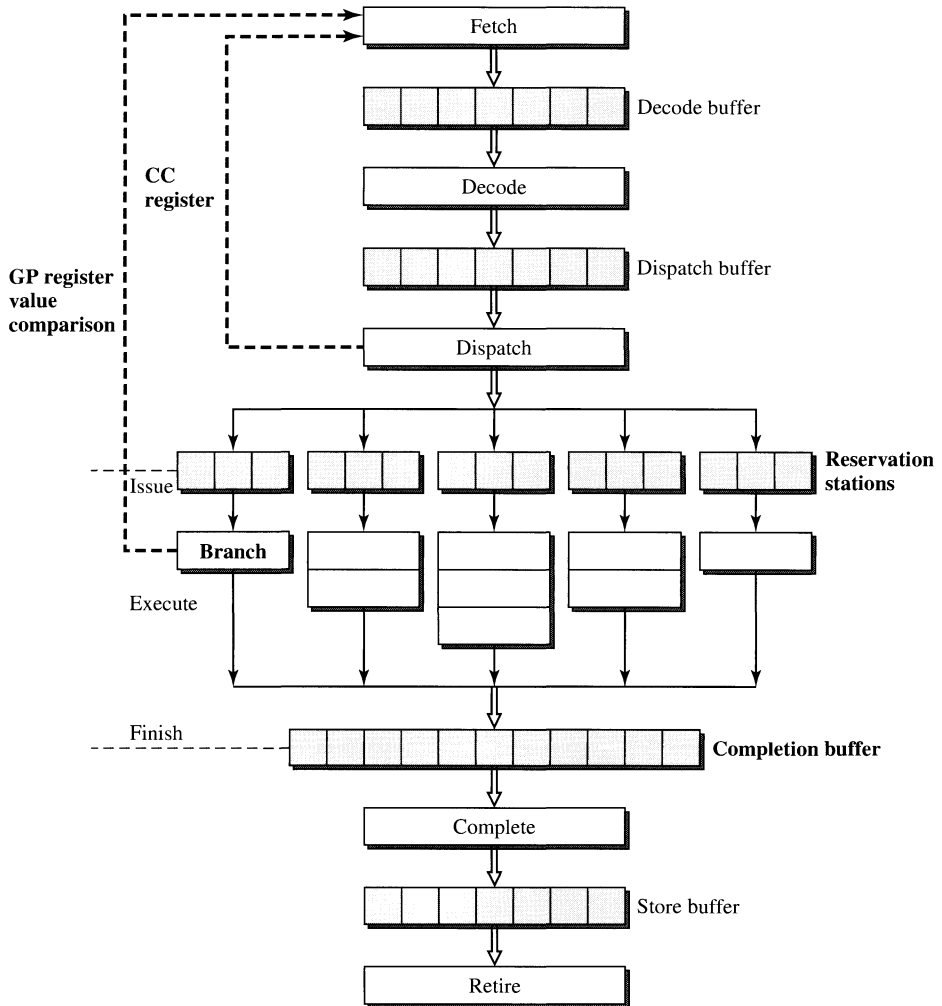
slots as in the scalar pipeline, but the number of empty instruction slots must be multiplied by the width of the machine. For example, for a four-wide machine the total penalty is 12 instruction “bubbles” in the superscalar pipeline. Also recall from Chapter 1, that such pipeline stall cycles effectively correspond to the *sequential bottleneck* of Amdahl’s law and rapidly and significantly reduce the actual performance from the potential peak performance.

For conditional branches, the actual number of stalled or penalty cycles can be dictated by either target address generation or condition resolution. Figure 5.3 illustrates the potential cycles that can be incurred by target address generation. The

**Figure 5.3**

Branch Target Address Generation Penalties.

actual number of penalty cycles is determined by the addressing modes of the branch instructions. For the PC-relative addressing mode, the branch target address can be generated during the fetch stage, resulting in a penalty of one cycle. If the register indirect addressing mode is used, the branch instruction must traverse the decode stage to access the register. In this case a two-cycle penalty is incurred. For register indirect with an offset addressing mode, the offset must be added after register access and a total three-cycle penalty can result. For unconditional branches, only the penalty due to target address generation is of concern. For conditional branches, branch condition resolution latency must also be considered.

**Figure 5.4**

Branch Condition Resolution Penalties.

Different methods for performing condition resolution can also lead to different penalties. Figure 5.4 illustrates two possible penalties. If condition code registers are used, and assuming that the relevant condition code register is accessed during the dispatch stage, then a penalty of two cycles will result. If the ISA permits the comparison of two general-purpose registers to generate the branch condition, then one more cycle is needed to perform an ALU operation on the contents of the two registers. This will result in a penalty of three cycles. For a conditional branch, depending on the addressing mode and condition resolution method used, either one of the penalties may be the critical one. For example, even if the PC-relative

addressing mode is used, a conditional branch that must access a condition code register will still incur a two-cycle penalty instead of the one-cycle penalty for target address generation.

Maximizing the volume of the instruction flow path is equivalent to maximizing the sustained instruction fetch bandwidth. To do this, the number of stall cycles in the fetch stage must be minimized. Recall that the total lost-opportunity cost is equal to the product of the number of penalty cycles and the width of a machine. For an n -wide machine each stalled cycle is equal to fetching n no-op instructions. The primary aim of instruction flow techniques is to minimize the number of such fetch stall cycles and/or to make use of these cycles to do potentially useful work. The current dominant approach to accomplishing this is via branch prediction which is the subject of Section 5.1.3.

5.1.3 Branch Prediction Techniques

Experimental studies have shown that the behavior of branch instructions is highly predictable. A key approach to minimizing branch penalty and maximizing instruction flow throughput is to speculate on both branch target addresses and branch conditions of branch instructions. As a static branch instruction is repeatedly executed at run time, its dynamic behavior can be tracked. Based on its past behavior, its future behavior can be effectively predicted. Two fundamental components of branch prediction are *branch target speculation* and *branch condition speculation*. With any speculative technique, there must be mechanisms to validate the prediction and to safely recover from any mispredictions. Branch misprediction recovery will be covered in Section 5.1.4.

Branch target speculation involves the use of a branch target buffer (BTB) to store previous branch target addresses. BTB is a small cache memory accessed during the instruction fetch stage using the instruction fetch address (PC). Each entry of the BTB contains two fields: the branch instruction address (BIA) and the branch target address (BTA). When a static branch instruction is executed for the first time, an entry in the BTB is allocated for it. Its instruction address is stored in the BIA field, and its target address is stored in the BTA field. Assuming the BTB is a fully associative cache, the BIA field is used for the associative access of the BTB. The BTB is accessed concurrently with the accessing of the I-cache. When the current PC matches the BIA of an entry in the BTB, a hit in the BTB results. This implies that the current instruction being fetched from the I-cache has been executed before and is a branch instruction. When a hit in the BTB occurs, the BTA field of the hit entry is accessed and can be used as the next instruction fetch address if that particular branch instruction is predicted to be taken; see Figure 5.5.

By accessing the BTB using the branch instruction address and retrieving the branch target address from the BTB all during the fetch stage, the speculative branch target address will be ready to be used in the next machine cycle as the new instruction fetch address if the branch instruction is predicted to be taken. If the branch instruction is predicted to be taken and this prediction turns out to be correct, then the branch instruction is effectively executed in the fetch stage, incurring no branch penalty. The nonspeculative execution of the branch instruction is still

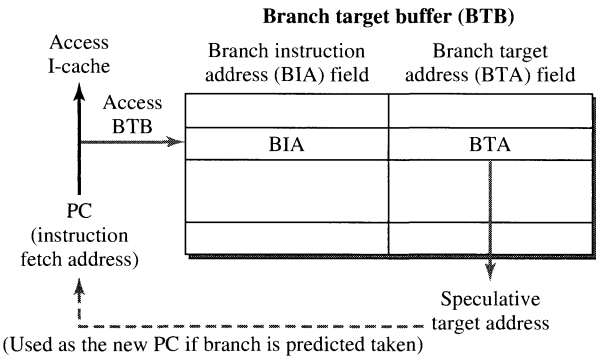
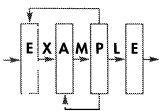


Figure 5.5
Branch Target Speculation Using a Branch Target Buffer.

performed for the purpose of validating the speculative execution. The branch instruction is still fetched from the I-cache and executed. The resultant target address and branch condition are compared with the speculative version. If they agree, then correct prediction was made; otherwise, misprediction has occurred and recovery must be initiated. The result from the nonspeculative execution is also used to update the content, i.e., the BTA field, of the BTB.

There are a number of ways to do branch condition speculation. The simplest form is to design the fetch hardware to be biased for not taken, i.e., to always predict not taken. When a branch instruction is encountered, prior to its resolution, the fetch stage continues fetching down the fall-through path without stalling. This form of minimal branch prediction is easy to implement but is not very effective. For example, many branches are used as loop closing instructions, which are mostly taken during execution except when exiting loops. Another form of prediction employs software support and can require ISA changes. For example, an extra bit can be allocated in the branch instruction format that is set by the compiler. This bit is used as a hint to the hardware to perform either predict not taken or predict taken depending on the value of this bit. The compiler can use branch instruction type and profiling information to determine the most appropriate value for this bit. This allows each static branch instruction to have its own specified prediction. However, this prediction is static in the sense that the same prediction is used for all dynamic executions of the branch. Such static software prediction technique is used in the Motorola 88110 [Diefendorf and Allen, 1992]. A more aggressive and dynamic form of prediction makes prediction based on the branch target address offset. This form of prediction first determines the relative offset between the address of the branch instruction and the address of the target instruction. A positive offset will trigger the hardware to predict not taken, whereas a negative offset, most likely indicating a loop closing branch, will trigger the hardware to predict taken. This branch offset-based technique is used in the original IBM RS/6000 design and has been adopted by other machines as well [Grohoski, 1990; Oehler and Groves, 1990]. The most common branch condition speculation technique



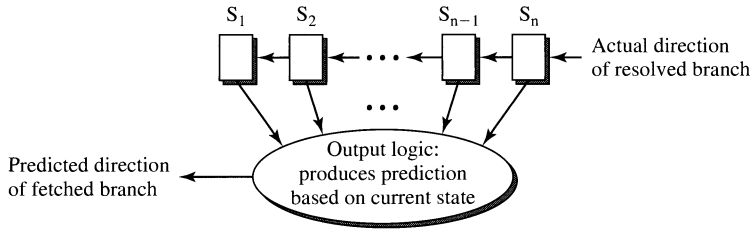


Figure 5.6

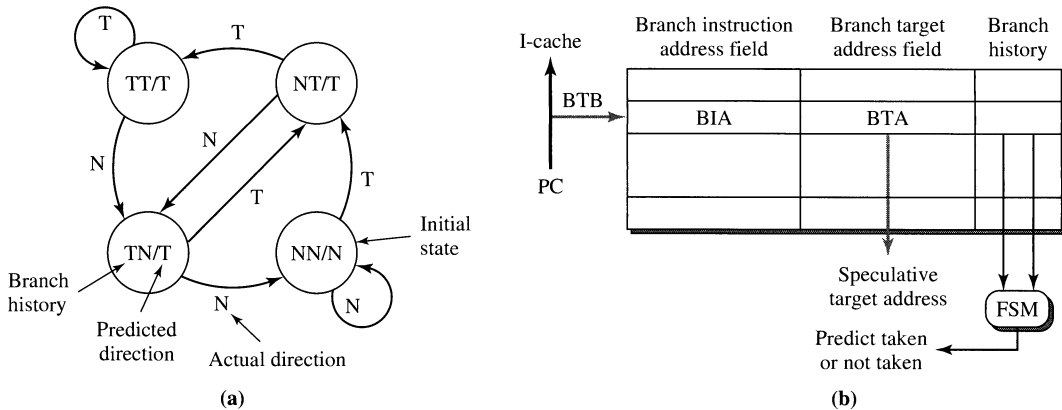
FSM Model for History-Based Branch Direction Predictors.

employed in contemporary superscalar machines is based on the history of previous branch executions.

History-based branch prediction makes a prediction of the branch direction, whether taken (T) or not taken (N), based on previously observed branch directions. This approach was first proposed by Jim Smith, who patented the technique on behalf of his employer, Control Data, and later published an important early study [Smith, 1981]. The assumption is that historical information on the direction that a static branch takes in previous executions can give helpful hints on the direction that it is likely to take in future executions. Design decisions for such type of branch prediction include how much history should be tracked and for each observed history pattern what prediction should be made. The specific algorithm for history-based branch direction prediction can be characterized by a finite state machine (FSM); see Figure 5.6. The n state variables encode the directions taken by the last n executions of that branch. Hence each state represents a particular history pattern in terms of a sequence of takens and not takens. The output logic generates a prediction based on the current state of the FSM. Essentially, a prediction is made based on the outcome of the previous n executions of that branch. When a predicted branch is finally executed, the actual outcome is used as an input to the FSM to trigger a state transition. The next state logic is trivial; it simply involves chaining the state variables into a shift register, which records the branch directions of the previous n executions of that branch instruction.

Figure 5.7(a) illustrates the FSM diagram of a typical 2-bit branch predictor that employs two history bits to track the outcome of two previous executions of the branch. The two history bits constitute the state variables of the FSM. The predictor can be in one of four states: NN, NT, TT, or TN, representing the directions taken in the previous two executions of the branch. The NN state can be designated as the initial state. An output value of either T or N is associated with each of the four states representing the prediction that would be made when a predictor is in that state. When a branch is executed, the actual direction taken is used as an input to the FSM, and a state transition occurs to update the branch history which will be used to do the next prediction.

The particular algorithm implemented in the predictor of Figure 5.7(a) is biased toward predicting branches to be taken; note that three of the four states

**Figure 5.7**

History-Based Branch Prediction: (a) A 2-Bit Branch Predictor Algorithm; (b) Branch Target Buffer with an Additional Field for Storing Branch History Bits.

predict the branch to be taken. It anticipates either long runs of N's (in the NN state) or long runs of T's (in the TT state). As long as at least one of the two previous executions was a taken branch, it will predict the next execution to be taken. The prediction will only be switched to not taken when it has encountered two consecutive N's in a row. This represents one particular branch direction prediction algorithm; clearly there are many possible designs for such history-based predictors, and many designs have been evaluated by researchers.

To support history-based branch direction predictors, the BTB can be augmented to include a history field for each of its entries. The width, in number of bits, of this field is determined by the number of history bits being tracked. When a PC address hits in the BTB, in addition to the speculative target address, the history bits are retrieved. These history bits are fed to the logic that implements the next-state and output functions of the branch predictor FSM. The retrieved history bits are used as the state variables of the FSM. Based on these history bits, the output logic produces the 1-bit output that indicates the predicted direction. If the prediction is a taken branch, then this output is used to steer the speculative target address to the PC to be used as the new instruction fetch address in the next machine cycle. If the prediction turns out to be correct, then effectively the branch instruction has been executed in the fetch stage without incurring any penalty or stalled cycle.

A classic experimental study on branch prediction was done by Lee and Smith [1984]. In this study, 26 programs from six different types of workloads for three different machines (IBM 370, DEC PDP-11, and CDC 6400) were used. Averaged across all the benchmarks, 67.6% of the branches were taken while 32.4% were not taken. Branches tend to be taken more than not taken by a ratio of 2 to 1. With static branch prediction based on the op-code type, the prediction accuracy ranged from 55% to 80% for the six workloads. Using only 1 bit of history, history-based dynamic branch prediction achieved prediction accuracies ranging from 79.7% to

96.5%. With 2 history bits, the accuracies for the six workloads ranged from 83.4% to 97.5%. Continued increase of the number of history bits brought additional incremental accuracy. However, beyond four history bits there is a very minimal increase in the prediction accuracy. They implemented a four-way set associative BTB that had 128 sets. The averaged BTB hit rate was 86.5%. Combining prediction accuracy with the BTB hit rate, the resultant average prediction effectiveness was approximately 80%.

Another experimental study was done in 1992 at IBM by Ravi Nair using the RS/6000 architecture and Systems Performance Evaluation Cooperative (SPEC) benchmarks [Nair, 1992]. This was a very comprehensive study of possible branch prediction algorithms. The goal for branch prediction is to overlap the execution of branch instructions with that of other instructions so as to achieve zero-cycle branches or accomplish *branch folding*; i.e., branches are folded out of the critical latency path of instruction execution. This study performed an exhaustive search for optimal 2-bit predictors. There are 2^{20} possible FSMs of 2-bit predictors. Nair determined that many of these machines are uninteresting and pruned the entire design space down to 5248 machines. Extensive simulations are performed to determine the optimal (achieves the best prediction accuracy) 2-bit predictor for each of the benchmarks. The list of SPEC benchmarks, their best prediction accuracies, and the associated optimal predictors are shown in Figure 5.8.

In Figure 5.8, the states denoted with bold circles represent states in which the branch is predicted taken; the nonbold circles represent states that predict not taken. Similarly the bold edges represent state transitions when the branch is actually taken; the nonbold edges represent transitions corresponding to the branch

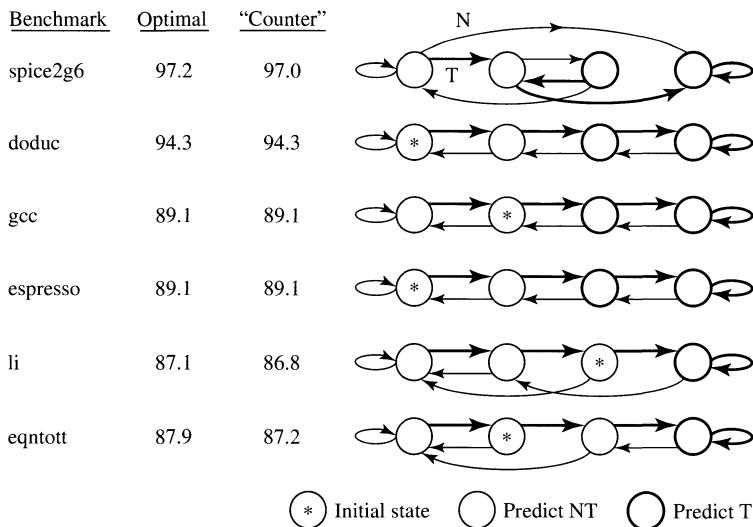


Figure 5.8

Optimal 2-Bit Branch Predictors for Six SPEC Benchmarks from the Nair Study.

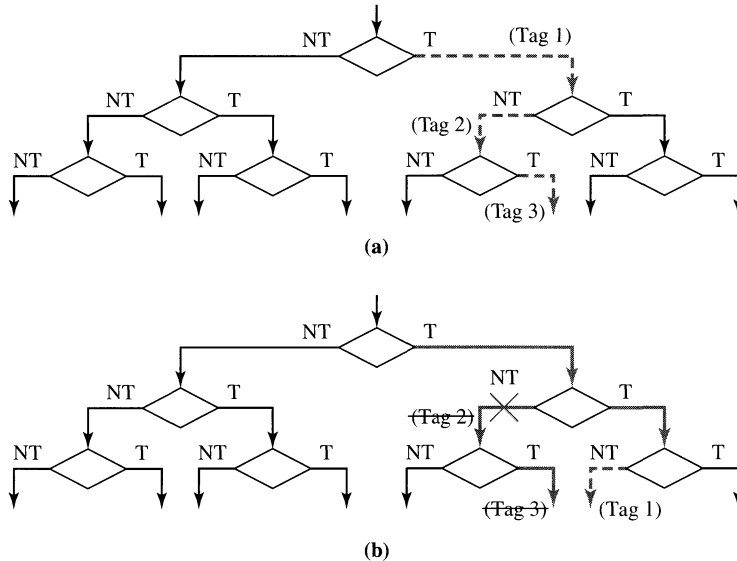
actually not taken. The state denoted with an asterisk indicates the initial state. The prediction accuracies for the optimal predictors of these six benchmarks range from 87.1% to 97.2%. Notice that the optimal predictors for *doduc*, *gcc*, and *espresso* are identical (disregarding the different initial state of the *gcc* predictor) and exhibit the behavior of a 2-bit up/down saturating counter. We can label the four states from left to right as 0, 1, 2, and 3, representing the four count values of a 2-bit counter. Whenever a branch is resolved taken, the count is incremented; and it is decremented otherwise. The two lower-count states predict a branch to be not taken, while the two higher-count states predict a branch to be taken. Figure 5.8 also provides the prediction accuracies for the six benchmarks if the 2-bit saturating counter predictor is used for all six benchmarks. The prediction accuracies for *spice2g6*, *li*, and *eqntott* only decrease minimally from their optimal values, indicating that the 2-bit saturating counter is a good candidate for general use on all benchmarks. In fact, the 2-bit saturating counter, originally invented by Jim Smith, has become a popular prediction algorithm in real and experimental designs.

The same study by Nair also investigated the effectiveness of counter-based predictors. With a 1-bit counter as the predictor, i.e., remembering the direction taken last time and predicting the same direction for the next time, the prediction accuracies ranged from 82.5% to 96.2%. As we have seen in Figure 5.8, a 2-bit counter yields an accuracy range of 86.8% to 97.0%. If a 3-bit counter is used, the increase in accuracy is minimal; accuracies range from 88.3% to 97.0%. Based on this study, the 2-bit saturating counter appears to be a very good choice for a history-based predictor. Direct-mapped branch history tables are assumed in this study. While some programs, such as *gcc*, have more than 7000 conditional branches, for most programs, the branch penalty due to aliasing in finite-sized branch history tables levels out at about 1024 entries for the table size.

5.1.4 Branch Misprediction Recovery

Branch prediction is a speculative technique. Any speculative technique requires mechanisms for validating the speculation. Dynamic branch prediction can be viewed as consisting of two interacting engines. The leading engine performs speculation in the front-end stages of the pipeline, while a trailing engine performs validation in the later stages of the pipeline. In the case of misprediction the trailing engine also performs recovery. These two aspects of branch prediction are illustrated in Figure 5.9.

Branch speculation involves predicting the direction of a branch and then proceeding to fetch along the predicted path of control flow. While fetching from the predicted path, additional branch instructions may be encountered. Prediction of these additional branches can be similarly performed, potentially resulting in speculating past multiple conditional branches before the first speculated branch is resolved. Figure 5.9(a) illustrates speculating past three branches with the first and the third branches being predicted taken and the second one predicted not taken. When this occurs, instructions from three speculative basic blocks are now resident in the machine and must be appropriately identified. Instructions from each speculative basic block are given the same identifying tag. In the example of

**Figure 5.9**

Two Aspects of Branch Prediction: (a) Branch Speculation; (b) Branch Validation/Recovery.

Figure 5.9(a), three distinct tags are used to identify the instructions from the three speculative basic blocks. A tagged instruction indicates that it is a speculative instruction, and the value of the tag identifies which basic block it belongs to. As a speculative instruction advances down the pipeline stages, the tag is also carried along. When speculating, the instruction addresses of all the speculated branch instructions (or the next sequential instructions) must be buffered in the event that recovery is required.

Branch validation occurs when the branch is executed and the actual direction of a branch is resolved. The correctness of the earlier prediction can then be determined. If the prediction turns out to be correct, the speculation tag is deallocated and all the instructions associated with that tag become nonspeculative and are allowed to complete. If a misprediction is detected, two actions are required; namely, the incorrect path must be terminated, and fetching from a new correct path must be initiated. To initiate a new path, the PC must be updated with a new instruction fetch address. If the incorrect prediction was a not-taken prediction, then the PC is updated with the computed branch target address. If the incorrect prediction was a taken prediction, then the PC is updated with the sequential (fall-through) instruction address, which is obtained from the previously buffered instruction address when the branch was predicted taken. Once the PC has been updated, fetching of instructions resumes along the new path, and branch prediction begins anew. To terminate the incorrect path, speculation tags are used. All the tags that are associated with the mispredicted branch are used to identify the

instructions that must be eliminated. All such instructions that are still in the decode and dispatch buffers as well as those in reservation station entries are invalidated. Reorder buffer entries occupied by these instructions are deallocated. Figure 5.9(b) illustrates this validation/recovery task when the second of the three predictions is incorrect. The first branch is correctly predicted, and therefore instructions with Tag 1 become nonspeculative and are allowed to complete. The second prediction is incorrect, and all the instructions with Tag 2 and Tag 3 must be invalidated and their reorder buffer entries must be deallocated. After fetching down the correct path, branch prediction can begin once again, and Tag 1 is used again to denote the instructions in the first speculative basic block. During branch validation, the associated BTB entry is also updated.

We now use the PowerPC 604 superscalar microprocessor to illustrate the implementation of dynamic branch prediction in a real superscalar processor. The PowerPC 604 is a four-wide superscalar capable of fetching, decoding, and dispatching up to four instructions in every machine cycle [IBM Corp., 1994]. Instead of a single unified BTB, the PowerPC 604 employs two separate buffers to support branch prediction, namely, the branch target address cache (BTAC) and the branch history table (BHT); see Figure 5.10. The BTAC is a 64-entry fully associative cache that stores the branch target addresses, while the BHT, a 512-entry direct-mapped table, stores the history bits of branches. The reason for this separation will become clear shortly.

Both the BTAC and the BHT are accessed during the fetch stage using the current instruction fetch address in the PC. The BTAC responds in one cycle; however, the BHT requires two cycles to complete its access. If a hit occurs in the BTAC, indicating the presence of a branch instruction in the current fetch group, a predict taken occurs and the branch target address retrieved from the BTAC is used in the next fetch cycle. Since the PowerPC 604 fetches four instructions in a fetch cycle, there can be multiple branches in the fetch group. Hence, the BTAC entry indexed by the fetch address contains the branch target address of the first branch instruction in the fetch group that is predicted to be taken. In the second cycle, or during the decode stage, the history bits retrieved from the BHT are used to generate a history-based prediction on the same branch. If this prediction agrees with the taken prediction made by the BTAC, the earlier prediction is allowed to stand. On the other hand, if the BHT prediction disagrees with the BTAC prediction, the BTAC prediction is annulled and fetching from the fall-through path, corresponding to predict not taken, is initiated. In essence, the BHT prediction can overrule the BTAC prediction. As expected, in most cases the two predictions agree. In some cases, the BHT corrects the wrong prediction made by the BTAC. It is possible, however, for the BHT to erroneously change the correct prediction of the BTAC; this occurs very infrequently. When a branch is resolved, the BHT is updated; and based on its updated content the BHT in turn updates the BTAC by either leaving an entry in the BTAC if it is to be predicted taken the next time, or deleting an entry from the BTAC if that branch is to be predicted not taken the next time.

The PowerPC 604 has four entries in the reservation station that feeds the branch execution unit. Hence, it can speculate past up to four branches; i.e., there

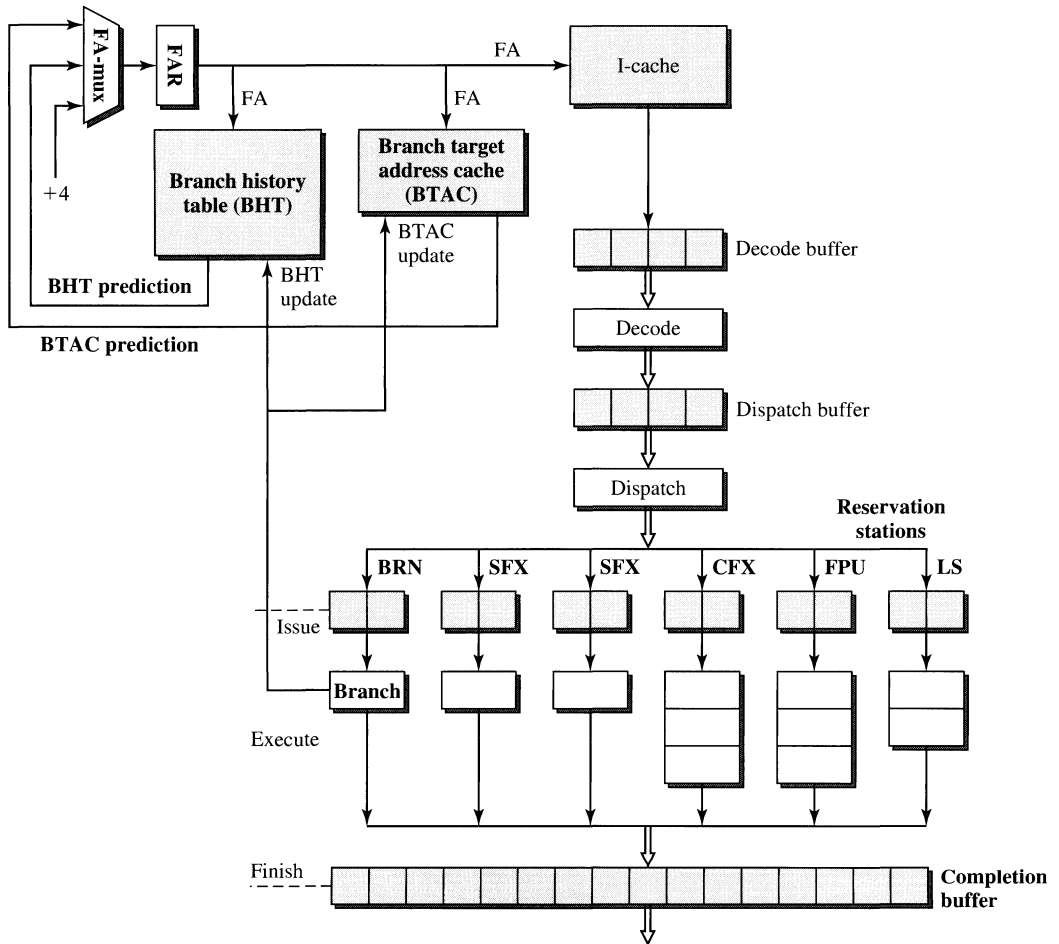


Figure 5.10

Branch Prediction in the PowerPC 604 Superscalar Microprocessor.

can be a maximum of four speculative branches present in the machine. To denote the four speculative basic blocks involved, a 2-bit tag is used to identify all speculative instructions. After a branch resolves, branch validation takes place and all speculative instructions either are made nonspeculative or are invalidated via the use of the 2-bit tag. Reorder buffer entries occupied by misspeculated instructions are deallocated. Again, this is performed using the 2-bit tag.

5.1.5 Advanced Branch Prediction Techniques

The dynamic branch prediction schemes discussed thus far have a number of limitations. Prediction for a branch is made based on the limited history of only that particular static branch instruction. The actual prediction algorithm does not take

into account the dynamic context within which the branch is being executed. For example, it does not make use of any information on the particular control flow path taken in arriving at that branch. Furthermore the same fixed algorithm is used to make the prediction regardless of the dynamic context. It has been observed experimentally that the behavior of certain branches is strongly correlated with the behavior of other branches that precede them during execution. Consequently more accurate branch prediction can be achieved with algorithms that take into account the branch history of other correlated branches and that can adapt the prediction algorithm to the dynamic branching context.

In 1991, Yeh and Patt proposed a two-level adaptive branch prediction technique that can potentially achieve better than 95% prediction accuracy by having a highly flexible prediction algorithm that can adapt to changing dynamic contexts [Yeh and Patt, 1991]. In previous schemes, a single branch history table is used and indexed by the branch address. For each branch address there is only one relevant entry in the branch history table. In the two-level adaptive scheme, a set of history tables is used. These are identified as the pattern history table (PHT); see Figure 5.11. Each branch address indexes to a set of relevant entries; one of these entries is then selected based on the dynamic branching context. The context is determined by a specific pattern of recently executed branches stored in a branch history shift register (BHSR); see Figure 5.11. The content of the BHSR is used to index into the PHT to select one of the relevant entries. The content of this entry is then used as the state for the prediction algorithm FSM to produce a prediction. When a branch is resolved, the branch result is used to update both the BHSR and the selected entry in the PHT.

The two-level adaptive branch prediction technique actually specifies a framework within which many possible designs can be implemented. There are two options

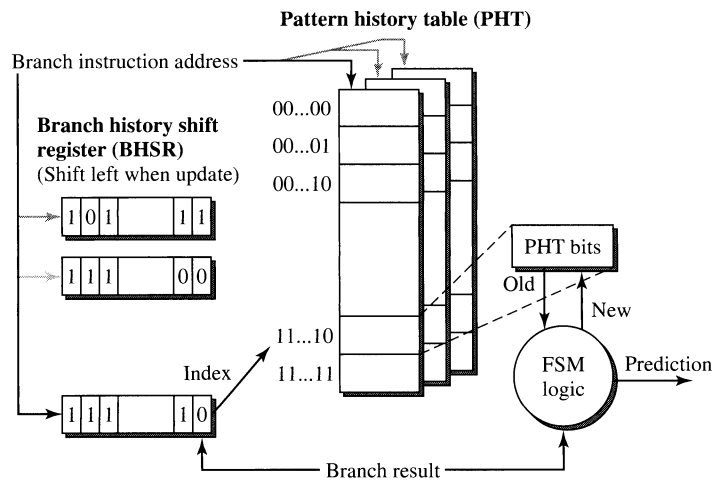


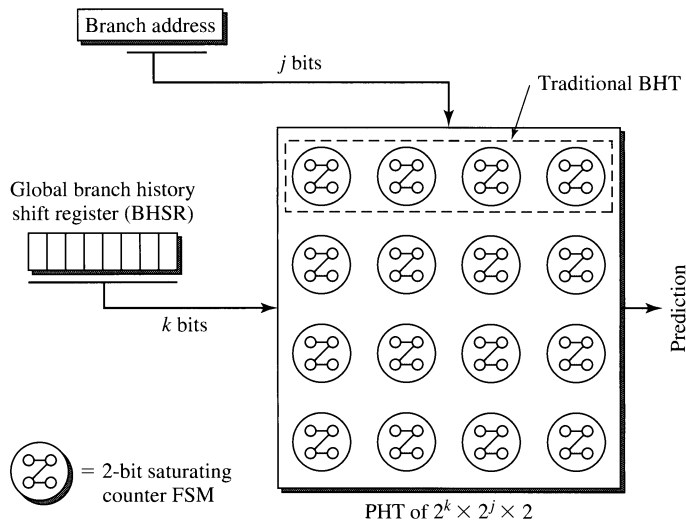
Figure 5.11
Two-Level Adaptive Branch Prediction of Yeh and Patt.
Source: Yeh and Patt, 1991.

to implementing the BHSR: *global* (G) and *individual* (P). The global implementation employs a single BHSR of k bits that tracks the branch directions of the last k dynamic branch instructions in program execution. These can involve any number (1 to k) of static branch instructions. The individual (called *per-branch* by Yeh and Patt) implementation employs a set of k -bit BHSRs as illustrated in Figure 5.11, one of which is selected based on the branch address. Essentially the global BHSR is shared by all static branches, whereas with individual BHSRs each BHSR is dedicated to each static branch or a subset of static branches if there is address aliasing when indexing into the set of BHSRs using the branch address. There are three options to implementing the PHT: *global* (g), *individual* (p), or *shared* (s). The global PHT uses a single table to support the prediction of all static branches. Alternatively, individual PHTs can be used in which each PHT is dedicated to each static branch (p) or a small subset of static branches (s) if there is address aliasing when indexing into the set of PHTs using the branch address. A third dimension to this design space involves the implementation of the actual prediction algorithm. When a history-based FSM is used to implement the prediction algorithm, Yeh and Patt identified such schemes as *adaptive* (A).

All possible implementations of the two-level adaptive branch prediction can be classified based on these three dimensions of design parameters. A given implementation can then be denoted using a three-letter notation; e.g., GAs represents a design that employs a single global BHSR, an adaptive prediction algorithm, and a set of PHTs with each being shared by a number of static branches. Yeh and Patt presented three specific implementations that are able to achieve a prediction accuracy of 97% for their given set of benchmarks:

- GAg: (1) BHSR of size 18 bits; (1) PHT of size $2^{18} \times 2$ bits.
- PAG: (512 $\times$ 4) BHSRs of size 12 bits; (1) PHT of size $2^{12} \times 2$ bits.
- PAs: (512 $\times$ 4) BHSRs of size 6 bits; (512) PHTs of size $2^6 \times 2$ bits.

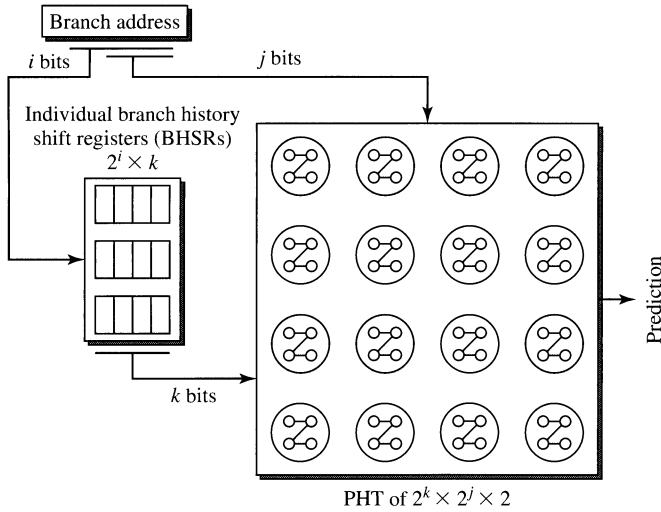
All three implementations use an adaptive (A) predictor that is a 2-bit FSM. The first implementation employs a global BHSR (G) of 18 bits and a global PHT (g) with 2^{18} entries indexed by the BHSR bits. The second implementation employs 512 sets (four-way set-associative) of 12-bit BHSRs (P) and a global PHT (g) with 2^{12} entries. The third implementation also employs 512 sets of four-way set-associative BHSRs (P), but each is only 6 bits wide. It also uses 512 PHTs (s), each having 2^6 entries indexed by the BHSR bits. Both the 512 sets of BHSRs and the 512 PHTs are indexed using 9 bits of the branch address. Additional branch address bits are used for the set-associative access of the BHSRs. The 512 PHTs are direct-mapped, and there can be aliasing, i.e., multiple branch addresses sharing the same PHT. From experimental data, such aliasing had minimal impact on degrading the prediction accuracy. Achieving greater than 95% prediction accuracy by the two-level adaptive branch prediction schemes is quite impressive; the best traditional prediction techniques can only achieve about 90% prediction accuracy. The two-level adaptive branch prediction approach has been adopted by a number of real designs, including the Intel Pentium Pro and the AMD/NexGen Nx686.

**Figure 5.12**

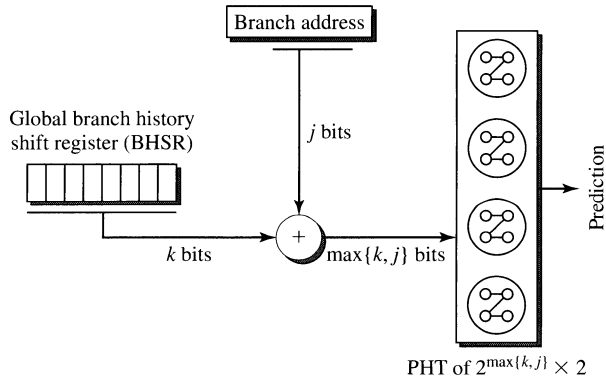
Correlated Branch Predictor with Global BHSR and Shared PHTs (GAs).

Following the original Yeh and Patt proposal, other studies by McFarling [1993], Young and Smith [1994], and Gloy et al. [1995] have gained further insights into two-level adaptive, or more recently called *correlated*, branch predictors. Figure 5.12 illustrates a correlated branch predictor with a global BHSR (G) and a shared PHT (s). The 2-bit saturating counter is used as the predictor FSM. The global BHSR tracks the directions of the last k dynamic branches and captures the dynamic control flow context. The PHT can be viewed as a single table containing a two-dimensional array, with 2^j columns and 2^k rows, of 2-bit predictors. If the branch address has n bits, a subset of j bits is used to index into the PHT to select one of the 2^j columns. Since j is less than n , some aliasing can occur where two different branch addresses can index into the same column of the PHT. Hence the designation of *shared* PHT. The k bits from the BHSR are used to select one of the 2^k entries in the selected column. The 2 history bits in the selected entry are used to make a history-based prediction. The traditional branch history table is equivalent to having only one row of the PHT that is indexed only by the j bits of the branch address, as illustrated in Figure 5.12 by the dashed rectangular block of 2-bit predictors in the first row of the PHT.

Figure 5.13 illustrates a correlated branch predictor with individual, or per-branch, BHSRs (P) and the same shared PHT (s). Similar to the GAs scheme, the PAs scheme also uses j bits of the branch address to select one of the 2^j columns of the PHT. However, i bits of the branch address, which can overlap with the j bits used to access the PHT, are used to index into a set of BHSRs. Depending on the branch address, one of the 2^i BHSRs is selected. Hence, each BHSR is associated with one particular branch address, or a set of branch addresses if there is aliasing. Essentially, instead of using a single BHSR to provide the dynamic control flow context for all static branches, multiple BHSRs are used to provide distinct dynamic control flow contexts for different subsets of static branches. This adds

**Figure 5.13**

Correlated Branch Predictor with Individual BHSRs and Shared PHTs (PAs).

**Figure 5.14**

The *gshare* Correlated Branch Predictor of McFarling.

Source: McFarling, 1993.

flexibility in tracking and exploiting correlations between different branch instructions. Each BHSR tracks the directions of the last k dynamic branches belonging to the same subset of static branches. Both the GAs and the PAs schemes require a PHT of size $2^k \times 2^j \times 2$ bits. The GAs scheme has only one k -bit BHSR whereas the PAs scheme requires 2^i k -bit BHSRs.

A fairly efficient correlated branch predictor called *gshare* was proposed by Scott McFarling [1993]. In this scheme, j bits from the branch address are “hashed” (via bitwise XOR function) with the k bits from a global BHSR; see Figure 5.14. The resultant $\max\{k, j\}$ bits are used to index into a PHT of size $2^{\max\{k, j\}} \times 2$ bits to

select one of the $2^{\max\{k,j\}}$ 2-bit branch predictors. The *gshare* scheme requires only one k -bit BHSR and a much smaller PHT, yet achieves comparable prediction accuracy to other correlated branch predictors. This scheme is used in the DEC Alpha 21264 4-way superscalar microprocessor [Keller, 1996].

5.1.6 Other Instruction Flow Techniques

The primary objective for instruction flow techniques is to supply as many useful instructions as possible to the execution core of the processor in every machine cycle. The two major challenges deal with conditional branches and taken branches. For a wide superscalar processor, to provide adequate conditional branch throughput, the processor must very accurately predict the outcomes and targets of multiple conditional branches in every machine cycle. For example, in a fetch group of four instructions, it is possible that all four instructions are conditional branches. Ideally one would like to use the addresses of all four instructions to index into a four-ported BTB to retrieve the history bits and target addresses of all four branches. A complex predictor can then make an overall prediction based on all the history bits. Speculative fetching can then proceed based on this prediction. Techniques for predicting multiple branches in every cycle have been proposed by Conte et al. [1995] as well as Rotenberg et al. [1996]. It is also important to ensure high accuracy in such predictions. Global branch history can be used in conjunction with per-branch history to achieve very accurate predictions. For those branches or sequences of branches that do not exhibit strongly biased branching behavior and therefore are not predictable, *dynamic eager execution* (DEE) has been proposed by Gus Uht [Uht and Sindagi, 1995]. DEE employs multiple PCs to simultaneously fetch from multiple addresses. Essentially the fetch stage pursues down multiple control flow paths until some branches are resolved, at which time some of the wrong paths are dynamically pruned by invalidating the instructions on those paths.

Taken branches are the second major obstacle to supplying enough useful instructions to the execution core. In a wide machine the fetch unit must be able to correctly process more than one taken branch per cycle, which involves predicting each branch's direction and target, and fetching, aligning, and merging instructions from multiple branch targets. An effective approach in alleviating this problem involves the use of a *trace cache* that was initially proposed by Eric Rotenberg [Rotenberg et al., 1996]. Since then, a form of trace caching has been implemented in Intel's most recent Pentium 4 superscalar microprocessor. Trace cache is a history-based fetch mechanism that stores dynamic instruction traces in a cache indexed by the fetch address and branch outcomes. These traces are assembled dynamically based on the dynamic branching behavior and can contain multiple nonconsecutive basic blocks. Whenever the fetch address hits in the trace cache, instructions are fetched from the trace cache rather than the instruction cache. Since a dynamic sequence of instructions in the trace cache can contain multiple taken branches but is stored sequentially, there is no need to fetch from multiple targets and no need for a multiported instruction cache or complex merging and aligning logic in the fetch stage. The trace cache can be viewed as doing dynamic

basic block reordering according to the dominant execution paths taken by a program. The merging and aligning can be done at completion time, when nonconsecutive basic blocks on a dominant path are first executed, to assemble a trace, which is then stored in one line of the trace cache. The goal is that once the trace cache is warmed up, most of the fetching will come from the trace cache instead of the instruction cache. Since the reordered basic blocks in the trace cache better match the dynamic execution order, there will be fewer fetches from nonconsecutive locations in the trace cache, and there will be an effective increase in the overall throughput of taken branches.

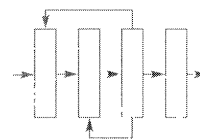
5.2 Register Data Flow Techniques

Register data flow techniques concern the effective execution of ALU (or register-register) type instructions in the execution core of the processor. ALU instructions can be viewed as performing the “real” work specified by the program, with control flow and load/store instructions playing the supportive roles of providing the necessary instructions and the required data, respectively. In the most ideal machine, branch and load/store instructions, being “overhead” instructions, should take no time to execute and the computation latency should be strictly determined by the processing of ALU instructions. The effective processing of these instructions is foundational to achieving high performance.

Assuming a load/store architecture, ALU instructions specify operations to be performed on source operands stored in registers. Typically an ALU instruction specifies a binary operation, two source registers where operands are to be retrieved, and a destination register where the result is to be placed. $R_i \leftarrow F_n(R_j, R_k)$ specifies a typical ALU instruction, the execution of which requires the availability of (1) F_n , the functional unit; (2) R_j and R_k , the two source operand registers; and (3) R_i , the destination register. If the functional unit F_n is not available, then a structural dependence exists that can result in a structural hazard. If one or both of the source operands in R_j and R_k are not available, then a hazard due to true data dependence can occur. If the destination register R_i is not available, then a hazard due to anti- and output dependences can occur.

5.2.1 Register Reuse and False Data Dependences

The occurrence of anti- and output dependences, or false data dependences, is due to the reuse of registers. If registers are never reused to store operands, then such false data dependences will not occur. The reuse of registers is commonly referred to as *register recycling*. Register recycling occurs in two different forms, one static and the other dynamic. The static form is due to optimization performed by the compiler and is presented first. In a typical compiler, toward the back end of the compilation process two tasks are performed: *code generation* and *register allocation*. The code generation task is responsible for the actual emitting of machine instructions. Typically the code generator assumes the availability of an unlimited number of symbolic registers in which it stores all the temporary data. Each symbolic register is used to store one value and is only written once, producing what is



CHAPTER 9

Gabriel H. Loh

Advanced Instruction Flow Techniques

CHAPTER OUTLINE

- 9.1 Introduction
- 9.2 Static Branch Prediction Techniques
- 9.3 Dynamic Branch Prediction Techniques
- 9.4 Hybrid Branch Predictors
- 9.5 Other Instruction Flow Issues and Techniques
- 9.6 Summary

References

Homework Problems

9.1 Introduction

In Chapter 5, it was stated that the instruction flow, or the processing of branches, provides an upper bound on the throughput of all subsequent stages. In particular, conditional branches in programs are a serious bottleneck to improving the rate of instruction flow and, hence, the performance of the processor. Before a conditional branch is resolved in a pipelined processor, it is unknown which instructions should follow the branch. To increase the number of instructions that execute in parallel, modern processors make a *branch prediction* and speculatively execute the instructions in the predicted path of program control flow. If the branch is discovered later on to have been mispredicted, actions are taken to recover the state of the processor to the point before the mispredicted branch, and execution is resumed along the correct path.

The penalty associated with mispredicted branches in modern pipelined processors has a great impact on performance. The performance penalty is increased

as the pipelines deepen and the number of outstanding instructions increases. For example, the AMD Athlon processor has 10 stages in the integer pipeline [Meyer, 1998], while the Intel NetBurst microarchitecture used in the Pentium 4 processor is “hyper-pipelined” with a 20-stage branch misprediction penalty [Hinton et al., 2001]. Several studies have suggested that the processor pipeline depth may continue to grow to 30 to 50 stages [Hartstein and Puzak, 2002; Hrishikesh et al., 2002]. Wide-issue superscalar processors further exacerbate the problem by creating a greater demand for instructions to execute. Despite the huge body of existing research in branch predictor design, these microarchitecture trends toward deeper and wider designs will continue to create a demand for more accurate branch prediction algorithms.

Processing conditional branches has two major components: predicting the branch direction and predicting the branch target. Sections 9.2 through 9.4, the bulk of this chapter, focus on the former problem of predicting whether a conditional branch is taken or not taken. Section 9.5 discusses the problem of branch target prediction and other issues related to effective instruction delivery.

Over the past two to three decades, there has been an incredible body of published research on the problem of predicting conditional branches and fetching instructions. The goal of this chapter is to take all these papers and distill the information down to the key ideas and concepts. Absolute comparisons such as whether one branch prediction algorithm is more accurate than another are difficult to make since such comparisons depend on a large number of assumptions such as the instruction set architecture, die area and clock frequency limitations, and choice of applications. The text makes note of techniques that have been implemented in commercial processors, but this does not necessarily imply that these algorithms are inherently better than some of the alternatives covered in this chapter. This chapter surveys a wide breadth of techniques with the aim of making the reader aware of the design issues and known methods in dealing with instruction flow.

The predictors described in this chapter are organized by how they make their predictions. Section 9.2 covers static branch predictors, that is, predictors that do not make use of any run-time information about branch behavior. Section 9.3 explains a wide variety of dynamic branch prediction algorithms, that is, predictors that can monitor branch behavior while the program is running and make future predictions based on these observations. Section 9.4 describes hybrid branch predictors that combine the strengths of multiple simpler predictors to form a better overall predictor.

9.2 Static Branch Prediction Techniques

Static branch prediction algorithms tend to be very simple and by definition do not incorporate any feedback from the run-time environment. This characteristic is both the strength and weakness of static prediction algorithms. By not paying any attention to the dynamic run-time behavior of a program, the branch prediction is incapable of adapting to changes in branch outcome patterns. These patterns may vary based on the input set for the program or different phases of a program’s execution.

The advantage of static branch prediction techniques is that they are very simple to implement, and they require very little hardware resources. Static branch prediction algorithms are of less interest in the context of future-generation, large transistor budget, very large-scale integration (VLSI) processors because the additional area for more effective dynamic branch predictors can be afforded. Nevertheless, static branch predictors may still be used as components in more complex hybrid branch predictors or as a simpler fallback predictor when no other prediction information is available.

Profile-based static prediction can achieve better performance than simpler *rule-based* algorithms. The key assumption underlying profile-based approaches is that the actual run-time behavior of a program can be approximated by different runs of the program on different data sets. In addition to the branch outcome statistics of sample executions, profile-based algorithms may also take advantage of information that is available at compile time such as the high-level structure of the program. The main disadvantage with profile-based techniques is that profiling must be part of the compilation phase of the program, and existing programs cannot take advantage of the benefits without being recompiled. If the branch behavior statistics collected from the training runs are not representative of the branch behavior in the actual run, then the profile-based predictions may not provide much benefit.

This section continues with a brief survey of some of the rule-based static branch prediction algorithms and then presents an overview of profile-based static branch prediction.

9.2.1 Single-Direction Prediction

The simplest branch prediction strategy is to predict that the direction of all branches will always go in the same direction (always taken or always not taken). Older pipelined processors, such as the Intel i486 [Intel Corporation, 1997], used the always-not-taken prediction algorithm. This trivial strategy simplifies the task of fetching instructions because the next instruction to fetch after a branch is always the next sequential instruction in the static order of the program. Apart from cache misses and branch mispredictions, the instructions will be fetched in an uninterrupted stream. Unfortunately, branches are more often taken than not taken. For integer benchmarks, branches are taken approximately 60% of the time [Uht, 1997].

The opposite strategy is to always predict that a branch will be taken. Although this usually achieves a higher prediction accuracy rate than an always-not-taken strategy, the hardware is more complex. The problem is that the branch target address is generally unavailable at the time the branch prediction is made. One solution is to simply stall the front end of the pipeline until the branch target has been computed. This wastes processing slots in the pipeline (i.e., this causes pipeline bubbles) and leads to reduced performance. If the branch instruction specifies its target in a PC-relative fashion, the destination address may be computed in as little as an extra cycle of delay. Such was the case for the early MIPS R-series pipelines [Kane and Heinrich, 1992]. In an attempt to recover some of the lost processing cycles due to the pipeline bubbles, a *branch delay slot* after the branch

instruction was architected into the ISA. That is, the instruction immediately following a branch instruction is always executed regardless of the outcome of the branch. In theory, the branch delay slots can then be filled with useful instructions, although studies have shown that compilers cannot effectively make use of all the available delay slots [McFarling and Hennessy, 1986]. Faster cycle times may introduce more pipeline stages before the branch target calculation has completed, thus increasing the number of wasted cycles.

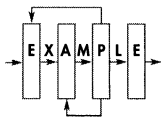
9.2.2 Backwards Taken/Forwards Not-Taken

A variation of the single-direction static prediction approach is the backwards taken/forwards not-taken (BTFNT) strategy. A *backwards branch* is a branch instruction that has a target with a lower address (i.e., one that comes earlier in the program). The rationale behind this heuristic is that the majority of backwards branches are loop branches, and since loops usually iterate many times before exiting, these branches are most likely to be taken. This approach does not require any modifications to the ISA since the sign of the target displacement is already encoded in the branch instruction. Many processors have used this prediction strategy; for example, the Intel Pentium 4 processor uses the BTFNT approach as a backup strategy when its dynamic predictor is unable to provide a prediction [Intel Corporation, 2003].

9.2.3 Ball/Larus Heuristics

Some instruction set architectures provide the compiler an interface through which *branch hints* can be made. These hints are encoded in the branch instructions, and an implementation of an ISA may choose to use these hints or not. The compiler can make use of these branch hints by inserting what it believes are the most likely outcomes of the branches based on high-level information about the structure of the program. This kind of static prediction is called *program-based* prediction.

There are branches in programs that almost always go in the same direction, but knowing the direction may require some high-level understanding of the programming language or the application itself. For example, consider the following code:



```
void * p = malloc (numBytes);
if (p == NULL)
    errorHandlingFunction( );
```

Except in very exceptional conditions, the call to `malloc` will return a valid pointer, and the following if-statement's condition will be false. Predicting the conditional branch that corresponds to this if-statement with a static prediction will result in perfect prediction rates (for all practical purposes).

Ball and Larus [1993] introduced a set of heuristics based on program structure to statically predict conditional branches. These rules are listed in Table 9.1. The heuristics make use of branch opcodes, the operands to branch instructions, and attributes of the instruction blocks that succeed the branch instructions in an attempt to make predictions based on the knowledge of common programming

Table 9.1

Ball and Larus's static branch prediction rules

Heuristic Name	Description
Loop branch	If the branch target is back to the head of a loop, predict taken.
Pointer	If a branch compares a pointer with NULL, or if two pointers are compared, predict in the direction that corresponds to the pointer being not NULL, or the two pointers not being equal.
Opcode	If a branch is testing that an integer is less than zero, less than or equal to zero, or equal to a constant, predict in the direction that corresponds to the test evaluating to false.
Guard	If the operand of the branch instruction is a register that gets used before being redefined in the successor block, predict that the branch goes to the successor block.
Loop exit	If a branch occurs inside a loop, and neither of the targets is the loop head, then predict that the branch does not go to the successor that is the loop exit.
Loop header	Predict that the successor block of a branch that is a loop header or a loop preheader is taken.
Call	If a successor block contains a subroutine call, predict that the branch goes to that successor block.
Store	If a successor block contains a store instruction, predict that the branch does not go to that successor block.
Return	If a successor block contains a return from subroutine instruction, predict that the branch does not go to that successor block.

idioms. In some situations, more than one heuristic may be applicable. For these situations, there is an ordering of the heuristics, and the first rule that is applicable is used. Ball and Larus evaluated all permutations of their rules to decide on the best ordering. Some of the rules capture the intuition that tests for exceptional conditions are rarely true (e.g., pointer and opcode rules), and some other rules are based on assumptions of common control flow patterns (the loop rules and the call/return rules).

9.2.4 Profiling

Profile-based static branch prediction involves executing an instrumented version of a program on sample input data, collecting statistics, and then feeding back the collected information to the compiler. The compiler makes use of the profile information to make static branch predictions that are inserted into the final program binary as branch hints.

One simple approach is to run the instrumented binary on one or more sample data sets and determine the frequency of taken branches for each static branch

instruction in the program. If more than one data set is used, then the measured frequencies can be weighted by the number of times each static branch was executed. The compiler inserts branch hints corresponding to the more frequently observed branch directions during the sample executions. If during the profiling run, a branch was observed to be taken more than 50% of the time, then the compiler would set the branch hint bit to predict-taken. In Fisher and Freudenberger [1992], such an experiment was performed, and it was found that for some benchmarks, different runs of a program were successful at predicting future runs on different data sets. In other cases, the success varied depending on how representative the sample data sets were.

The advantage of profile-based prediction techniques and the other static branch prediction algorithms is that they are very simple to implement in hardware. One disadvantage of profile-based prediction is that once the predictions are made, they are forever “set in stone” in the program binary. If an input set causes branching behaviors that are different from the training sets, performance will suffer. Additionally, the instruction set architecture must provide some interface to the programmer or compiler to insert branch hints.

Except for the always-taken and always-not-taken approaches, rule-based and profile-based branch prediction have the shortcoming that the branch instruction must be fetched from the instruction cache to be able to read the prediction embedded in the branch hint. Modern processors use multicycle pipelined instruction caches, and therefore the prediction for the next instruction must be available several cycles before the current instruction is fetched. In the following section, the dynamic branch prediction algorithms only make use of the address of the current branch and other information that is immediately available.

9.3 Dynamic Branch Prediction Techniques

Although static branch prediction techniques can achieve conditional branch prediction rates in the 70% to 80% range [Calder et al., 1997], if the profiling information is not representative of the actual run-time behavior, prediction accuracy can suffer greatly. Dynamic branch prediction algorithms take advantage of the run-time information available in the processor, and can react to changing branch patterns. Dynamic branch predictors typically achieve branch prediction rates in the range of 80% to 95% (for example, see McFarling [1993] and Yeh and Patt [1992]).

There are some branches that static prediction approaches cannot handle, but the branch behavior is still fundamentally very predictable. Consider a branch that is always taken during the first half of the program, and then is always not taken in the second half of the program. Profiling will reveal that the branch is taken 50% of the time, and any static prediction will result in a 50% prediction accuracy. On the other hand, if we simply predict that the branch will go in the same direction as the last time we encountered the branch, we can achieve nearly perfect prediction, with only a single misprediction at the halfway point of the program when the branch changes directions. Another situation where very predictable branches cannot be determined at compile time is where a branch’s direction depends on the program’s input. As an example, a program that performs matrix computations

may have different algorithms optimized for different sized matrices. Throughout the program, there may be branches that check the size of the matrix and then branch to the appropriate optimized code. For a given execution of this program, the matrix size is constant, and so these branches will have the same direction for the entire execution. By observing the run-time behavior, a dynamic branch predictor could easily predict all these branches. On the other hand, the compiler does not have any idea what size the matrices will be and is incapable of making much more than a blind guess.

Dynamic branch predictors may require a significant amount of chip area to implement, especially when more complex algorithms are used. For small processors, such as older-generation CPUs or processors targeted for embedded systems, the additional area for these prediction structures may simply be too expensive. For larger, future-generation, wide-issue superscalar processors, accurate conditional branch prediction is critical. Furthermore, these processors have much larger chip areas, and so considerable resources may be dedicated to the implementation of more sophisticated dynamic branch predictors. An additional benefit of dynamic branch prediction is that performance enhancements can be realized without profiling all the applications that one wishes to run, and recompilation is not needed so existing binary executables can benefit.

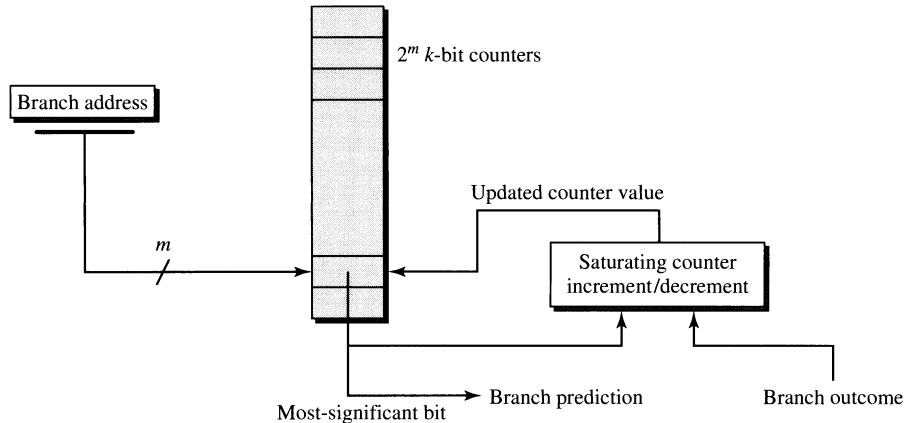
This section describes many of the dynamic branch prediction algorithms that have been published. Many of these prediction algorithms are important on their own, and some have even been implemented in commercial processors. In Section 9.4, we will also explore ways of composing more than one of these predictors into more powerful hybrid branch predictors.

This section has been divided into three parts based on the characteristics of the prediction algorithms. Section 9.3.1 covers several fundamental prediction schemes that are the basis for many of the more sophisticated algorithms. Section 9.3.2 describes predictors that address the branch aliasing problem. Section 9.3.3 covers prediction schemes that make use of a wider variety of information in making predictions.

9.3.1 Basic Algorithms

Most dynamic branch predictors have their roots in one or more of the basic algorithms described here.

9.3.1.1 Smith's Algorithm. The main idea behind the majority of dynamic branch predictors is that each time the processor discovers the true outcome of a branch (whether it is taken or not taken), it makes note of some form of context so that the next time it encounters the same situation, it will make the same prediction. An analogy for branch prediction is the problem of navigating in a car to get from one place to another where there are forks in the road. The driver just wants to keep driving as fast as she can, and so each time she encounters a fork, she can just guess a direction and keep on going. At the same time, her "copilot" (who happens to be slow at map reading) is trying to keep up. When he realizes that they made a wrong turn, he notifies the driver and then she will have to backtrack and then resume along the correct route.

**Figure 9.1**

Smith Predictor with a 2^m -entry Table of Saturating k -bit Counters.

If these two friends frequently drive in this area, the driver can do better than blindly guessing at each fork in the road. She might notice that they always end up making a right turn at the intersection with the pizza shop and always make a left at the supermarket. These landmarks form the context for the driver's predictions. In a similar fashion, dynamic branch predictors make note of context (in the form of branch history), and then make their predictions based on this information.

Smith's algorithm [1981] is one of the earliest proposed dynamic branch direction prediction algorithms, and one of the simplest. The predictor consists of a table that records for each branch whether or not the previous instances were taken or not taken. This is analogous to our driver keeping track in her head of the cross streets for each intersection and remembering if they went left or right. The cross streets correspond to the branch addresses, and the left/right decisions correspond to taken/not-taken branch outcomes. Because the predictor tracks whether a branch is in a mostly taken mode or a mostly not-taken mode, the name *bimodal* predictor is also commonly used for the Smith predictor.

The Smith predictor consists of a table of 2^m counters, where each counter tracks the past branch directions. Since there are only 2^m entries, the branch address [program counter (PC)] is hashed down to m bits.¹ Each counter in the table has a width of k bits. The most-significant bit of the counter is used for the branch direction prediction. If the most-significant bit is a one, then the branch is predicted to be taken; if the most significant bit is a zero, the branch is predicted to be not-taken. Figure 9.1 illustrates the hardware for Smith's algorithm. The notation Smith_k means Smith's algorithm with $k = \kappa$.

¹In his 1981 paper, Smith proposed an exclusive-OR hashing function, although most modern implementations use a simple $(\text{PC} \bmod 2^m)$ hashing function which requires no logic to implement. Typically, a few of the least-significant bits are ignored due to the fact that for an ISA with instruction word sizes that are powers of two, the lower bits will always be zero.

After a branch has resolved and its true direction is known, the counter is updated depending on the branch outcome. If the branch was taken, then the counter is incremented only if the current value is less than the maximum possible. For instance, a k -bit counter will *saturate* at $2^k - 1$. If the branch was not taken, then the counter is decremented if the current value is greater than zero.² This simple finite state machine is also called a *saturating k -bit counter*, or an *up-down counter*. The counter will have a higher value if the corresponding branch was often taken in the last several encounters of this branch. The counter will tend toward lower values when the recent branches have mostly been not taken. The case of Smith's algorithm when $k = 1$ simply keeps track of the last outcome of a branch that mapped to the counter.

Some branches are predominantly biased toward one direction. A branch at the end of a for loop is usually taken, except for the case of the loop exit. This one exceptional case is called an *anomalous decision*. The outcomes of several of the most recent branches to map to the same counter can be used if $k > 1$. By using the histories of several recent branches, the counter will not be thrown off by a single anomalous decision. The additional bits add some *hysteresis* to the predictor's state. Smith also calls this *inertia*.

Returning to the analogy of the driver, it may be the case that she almost always makes a left turn at a particular intersection, but most recently she ended up having to make a right turn instead because her and her friend had to go to the hospital due to an emergency. If our driver only remembered her most recent trip, then she would predict to make a right turn again the next time she was at this intersection. On the other hand, if she remembered the last several trips, she would realize that more often than not she ended up making a left turn. Using additional bits in the counter allows the predictor to effectively remember more history.

The 2-bit saturating counter (2bC) is used in many branch prediction algorithms. There are four possible states: 00, 01, 10, 11. States 00 and 01, called strongly not-taken (SN) and weakly not-taken (WN), respectively, provide predictions of not-taken. States 10 and 11, called weakly taken (WT) and strongly taken (ST), respectively, provide a taken-branch prediction. The reason states 00 and 11 are called "strong" is that the same outcome must have occurred multiple times to reach that state.

Figure 9.2 illustrates a short sequence of branches and the predictions made by Smith's algorithm for $k = 1$ (Smith₁) and $k = 2$ (Smith₂). Prior to the anomalous decision, both versions of Smith's algorithm predict the branches accurately. On the anomalous decision (branch C), both predictors mispredict. On the following branch D, Smith₁ mispredicts again because it only remembers the most recent branch and predicts in the same direction. This occurs despite the fact that the vast majority of prior branches were taken. On the other hand, Smith₂ makes the correct decision because its prediction is influenced by several of the most recent branches instead of the single most recent branch. For such anomalous decisions, Smith₁ makes two mispredictions while Smith₂ only errs once.

²The original paper presented the counter as using values from -2^{k-1} up to $2^{k-1} - 1$ in two's complement notation. The complement of the most-significant bit is then used as the branch direction prediction. The formulation presented here is used in the more recent literature.

Branch	Branch Direction	Smith ₁		Smith ₂	
		State	Prediction	State	Prediction
A	1	1	1	11	1
B	1	1	1	11	1
C	0	1	1 (misprediction)	11	1 (misprediction)
D	1	0	0 (misprediction)	10	1
E	1	1	1	11	1
F	1	1	1	11	1

Figure 9.2
A Comparison of a Smith₁ and a Smith₂ Predictor on a Sequence of Branches with a Single Anomalous Decision.

Practically every dynamic branch prediction algorithm published since Smith’s seminal paper uses saturating counters. For tracking branch directions, 2-bit counters provide better prediction accuracies than 1-bit counters due to the additional hysteresis. Adding a third bit only improves performance by a small increment. In many branch predictor designs, this incremental improvement is not worth the 50% increase in area of adding an additional bit to every 2-bit counter.

9.3.1.2 Two-Level Prediction Tables. Yeh and Patt [1991; 1992; 1993] and Pan et al. [1992] proposed variations of the same branch prediction algorithms called two-level adaptive branch prediction and correlation branch prediction, respectively. The two-level predictor employs two separate levels of branch history information to make the branch prediction.

Using the car navigation analogy, the Smith predictor parallel for driving was to remember what decision was made at each intersection. The car-navigation equivalent to the two-level predictor is for our driver to remember the exact sequence of turns made before arriving at the current intersection. For example, to drive from her apartment to the bank, our driver makes three turns: a left turn, another left, and then a right. To drive from the mall to the bank, she also makes three turns, but they are a right, a left, and then another right. If she finds herself at the bank and remembers that she most recently went right, left, and right, then she could guess that she just came from the mall and is heading home and make her next routing decision accordingly.

The *global-history* two-level predictor uses a history of the most recent branch outcomes. These outcomes are stored in the *branch history register* (BHR). The BHR is a shift register where the outcome of each branch is shifted into one end,

and the oldest outcome is shifted out of the other end and discarded. The branch outcomes are represented by zeros and ones, which correspond to not-taken and taken, respectively. Therefore, an h -bit branch history register records the h most recent branch outcomes. The branch history is the first level of the global-history two-level predictor.

The second level of the global-history two-level predictor is a table of saturating 2-bit counters (2bCs). This table is called the *pattern history table* (PHT). The PHT is indexed by a concatenation of a hash of the branch address with the contents of the BHR. This is analogous to our driver using a combination of the intersection as well as the most recent turn decisions in making her prediction. The counter in the indexed PHT entry provides the branch prediction in the same fashion as the Smith predictor (prediction is determined by the most-significant bit of the counter). Updates to the counter are also the same as for the Smith predictor counters: saturating increment on a taken branch, and saturating decrement on a not-taken branch.

Figure 9.3 shows the hardware organization of a sample global-history two-level predictor. This predictor uses the outcomes of the four most recent branch instructions and 2 bits from the branch address to form an index into a 64-entry PHT. With h bits of branch history and m bits of branch address, the PHT has 2^{h+m} entries. When using only m bits of branch address (where m is less than the total width of the PC), the branch address must be hashed down to m bits, similar to the Smith predictor. Note that in the example in Figure 9.3, this means that *any* branch address that ends in 01 will share the same entries as the branch depicted in the figure. Using the car navigation analogy again, this is similar to our driver remembering Elm Street as simply “Elm,” which may cause confusion when she encounters an Elm Road, Elm Lane, or Elm Boulevard. Note that this problem is not unique to the two-level predictor, and that it can affect the Smith predictor as well. Since the

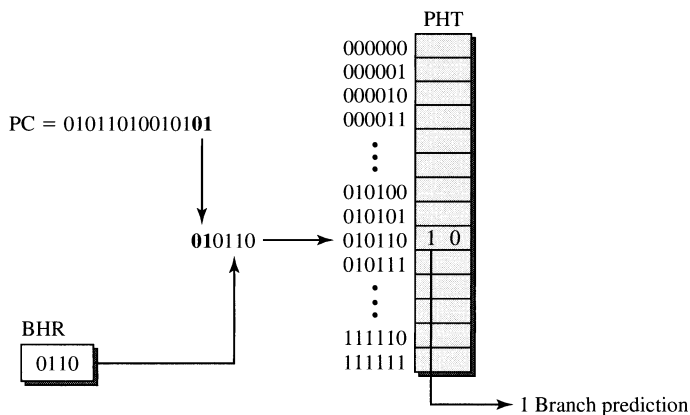


Figure 9.3

A Global-History Two-Level Predictor with a 4-bit Branch History Register.

Smith predictor does not use branch history, all its index bits are from the branch address, which reduces this branch conflict problem.

The size of the global-history two-level predictor depends on the total available hardware budget. For a 4K-byte budget, the PHT would have 16,384 entries (4,096 bytes times 4 two-bit counters per byte). In general, for an X K-byte budget, the PHT will contain $4X$ counters. There is a tradeoff between the number of branch address bits used and the length of the BHR, since the sum of their lengths must be equal to the number of index bits. Using more branch address bits reduces the branch conflict problem, whereas using more branch history allows the predictor to correlate against more complex branch history patterns. The optimal balance depends on many factors such as how the compiler arranges the code, the program being run, the instruction set architecture, and the input set to the program.

The intuition behind using the global branch history is that the behavior of a branch may be linked or *correlated* with a different earlier branch. For example, the branches may test conditions that involve the same variable. Another more common situation is that one branch may guard an instruction that modifies a variable that the second branch tests. Figure 9.4 shows a code segment where an if-statement (branch A) determines whether or not the variable x gets assigned a different value. Later in the program, another if-statement (branch C) tests the value of x . If A's condition was true, then x gets assigned the value of 3, and then C will evaluate $x \leq 0$ as false. On the other hand, if A's condition was false, then x retains its original value of 0, and C will evaluate $x \leq 0$ as true. The behavior (outcome) of the branch corresponding to if-statement C is strongly correlated to the outcome of if-statement A. A branch predictor that tracks the outcome of if-statement A could potentially achieve perfect prediction for the branch of if-statement C. Notice that there could be an intervening branch B that does not affect the outcome of C (that is, C is not correlated to B). Such irrelevant branches

```
x = 0;
if (someCondition) {           /* branch A */
    x = 3;
}
if (someOtherCondition) {      /* branch B */
    y += 19;
}
if (x <= 0) {                  /* branch C */
    doSomething( );
}
```

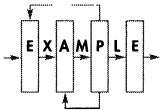


Figure 9.4

A Sample Code Segment with Correlated Branches.

increase the training time of global history predictors because the predictor must learn to ignore these irrelevant history bits.

Another variation of the two-level predictor is the *local-history* two-level predictor. Whereas global history tracks the outcomes of the last several branches encountered, local history tracks the outcomes of the last several encounters of only the current branch. Using the car navigation analogy again, our driver might make a right turn at a particular intersection during the week on her way to work, but on the weekends she makes left turns at the exact same intersection to go downtown. The turns she made to get to that intersection (i.e., the global history) might be the same regardless of the day of week. On the other hand, if she remembers that over the last several days she went R, R, R, L, then today is probably Sunday and she would predict that making a left turn is the correct decision. To remember a driving decision history for each intersection requires our driver to remember much more information than a simple global history, but there are some patterns that are easier to predict using such a local history.

To implement a local-history branch predictor, the single global BHR is replaced by one BHR per branch. The collection of BHRs form a *branch history table* (BHT). A global BHR is really a degenerate case where the BHT has only a single entry. The branch address is used to select one of the entries in the BHT, which provides the local history. The contents of the selected BHR are then combined with the PC in the same fashion as the global-history two-level predictor to index into the PHT. The most-significant bit of the counter provides the branch prediction, and the update of the counter is also the same as the Smith predictor. To update the history, the most recent branch outcome is shifted into the selected entry from the BHT.

Figure 9.5 shows the hardware organization for an example local-history two-level predictor. The BHT has eight entries and is indexed by the three least-significant bits of the branch address. The PHT in this example has 128 entries, which uses a

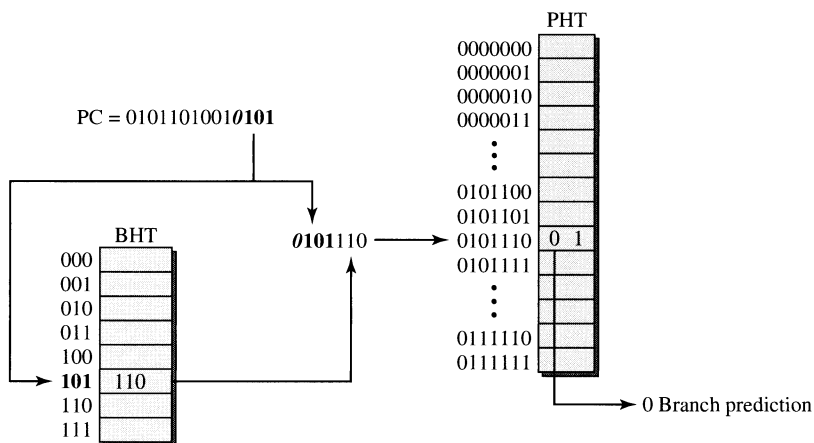
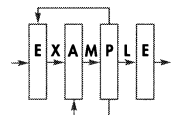


Figure 9.5

A Local-History Two-Level Predictor with an Eight-Entry BHT and a 3-bit History Length.



7-bit index ($\log_2 128 = 7$). Since the history length is 3 bits long, the other 4 index bits come from the branch address. These bits are concatenated together to select a counter from the PHT which provides the final prediction.

The tradeoffs in sizing a local-history two-level predictor are more complex than the case of the global-history predictor. In addition to balancing the number of history and address bits for the PHT index, there is also a tradeoff between the number of bits dedicated to the BHT and the number of bits dedicated to the PHT. In the BHT, there is also a balance between the number of entries and the width of each entry (i.e., the history length). A local-history two-level predictor with an L -entry BHT and an h -bit history and that uses m bits of the branch address for the PHT index requires a total size of $Lh + 2^{h+m+1}$ bits. The Lh bits are for the BHT, and the PHT has 2^{h+m} entries, each 2 bits wide (the +1 in the exponent).

Figure 9.6(a) shows an example predictor with an 8-entry BHT, a 4-bit history length, and a 64-entry PHT (only the first 16 entries are shown). The last four outcomes for the branch at address 0xC084 have been T, N, T, N. To select a BHR, we hash the branch address down to three bits by using the 3 least-significant bits of the address (100 in binary). Note that the selected branch history register's contents are 1010, which corresponds to a history of TNTN. With a 64-entry PHT, the size of the PHT index is 6 bits, of which 4 will be from the branch history. This

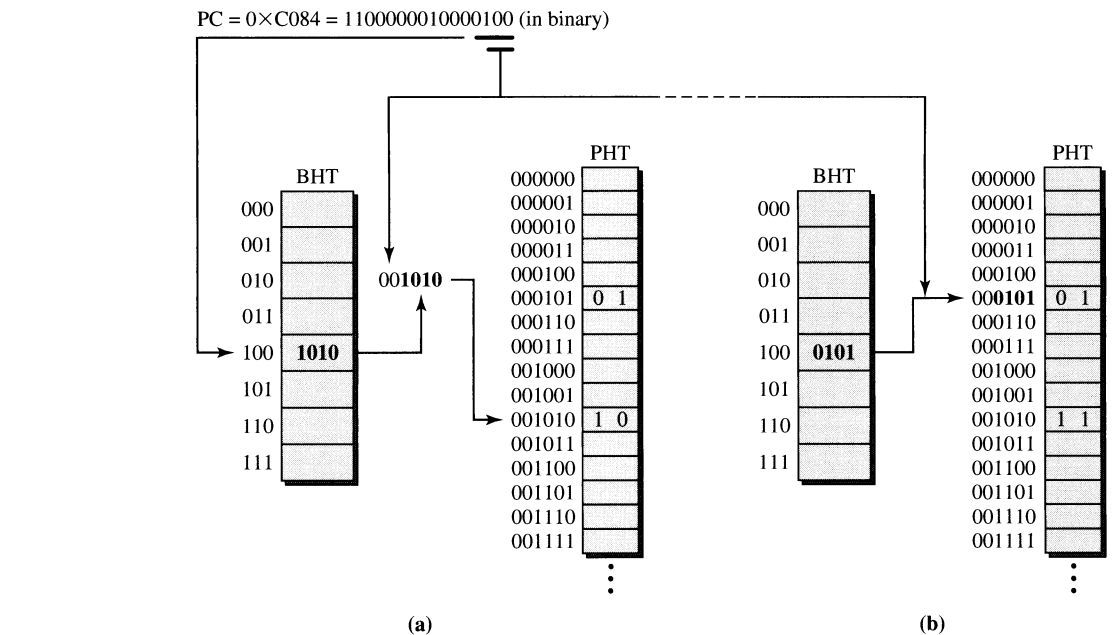


Figure 9.6 An Example Lookup on a Two-Level Branch Predictor: (a) Making the Prediction, and (b) After the Predictor Update.

leaves only 2 bits from the branch address. The concatenation of the branch address bits and the branch history selects one of the counters, whose most significant bit indicates a taken-branch prediction.

After the actual branch outcome has been computed during the execute stage of the pipeline, both the BHT and PHT need to be updated. Assuming that the actual outcome was a taken branch, a 1 is shifted into the BHR as shown in Figure 9.6(b). The PHT is updated as per the Smith predictor algorithm, and the 2-bit counter gets incremented from 10 to 11 (in binary). Note that for the next time the branch at 0xC084 is encountered, the BHR now contains the pattern 0101 which selects a different entry in the PHT. The Intel P6 microarchitecture uses a local-history two-level predictor with a 4-bit history length (see Chapter 7).

By tracking the behavior of each branch individually, a predictor can detect patterns that are *local* to a particular branch, like the alternating pattern shown in Figure 9.6. As a second example, consider a loop-closing branch with a short iteration count that exhibits the pattern 1110111011101 . . . , where again a 1 denotes a taken branch, and a 0 denotes a not-taken branch. By tracking the last several outcomes of only this particular branch, the PHT will quickly learn this pattern. Figure 9.7 shows a 16-entry PHT with the entries corresponding to predicting this pattern (no branch address bits are used for this PHT index). When the last four outcomes of this branch are 1101, the loop has not yet terminated and the next time the branch will again be taken. Every time the processor encounters the pattern 1101, the following branch is always taken. This results in incrementing the corresponding saturating counter at index 1101 every time this pattern occurs. After the pattern has occurred a few times, the PHT will predict taken because the counter indexed by 1101 will now remember (by having the state ST) that the

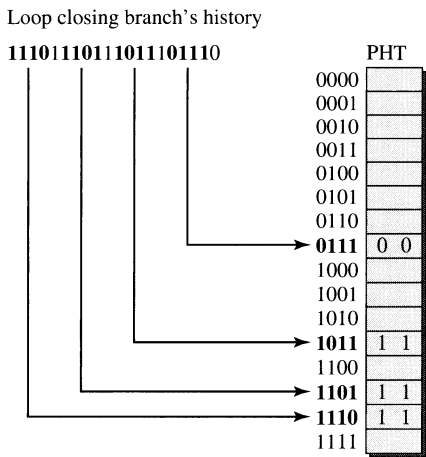
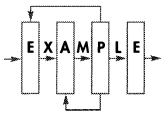
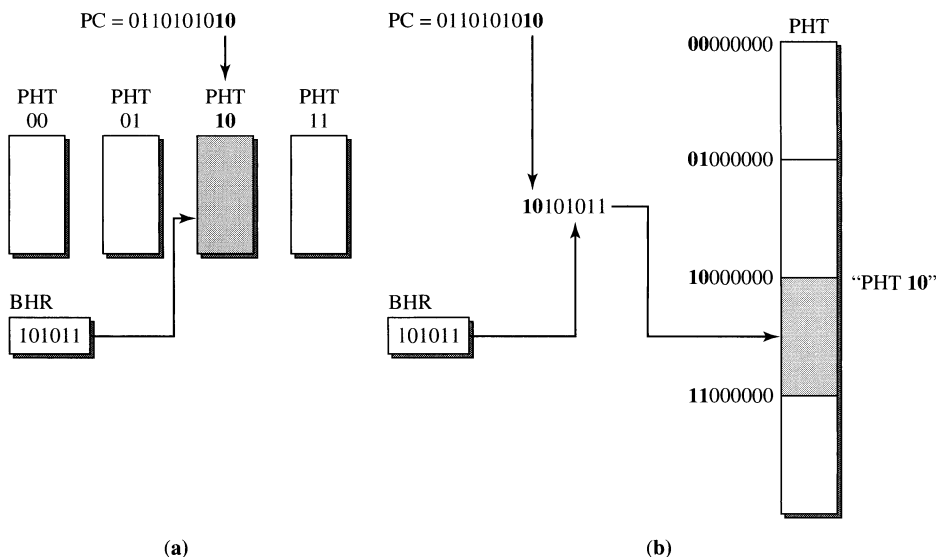


Figure 9.7
Local History Predictor Example.



**Figure 9.8**

Alternative Views of the PHT Organization: (a) A Collection of PHTs, and (b) A Single Monolithic PHT.

following instance of this branch will be taken. When the last four outcomes are 0111, the entry indexed by 0111 has a not-taken prediction stored (SN) state. The PHT basically learns a mapping of the form “when I see a history pattern of X, the outcome of the next branch is usually Y.”

Some texts and research papers view the two-level predictors as having multiple PHTs. The branch address hash selects one of the PHTs, and then the branch history acts as a subindex into the chosen PHT. This view of the two-level predictor is shown in Figure 9.8(a). The monolithic PHT shown in Figure 9.8(b) is actually equivalent. This chapter uses the monolithic view of the PHT because it reduces the PHT design tradeoffs down to a conceptually simpler problem of deciding how to allocate the bits of the index (i.e., how many index bits come from the PC and how long is the history length?).

Yeh and Patt [1993] also introduced a third variation that utilizes a BHT that uses an arbitrary hashing function to divide the branches into different *sets*. Each group shares a single BHR. Instead of using the least-significant bits of the branch address to select a BHR from the BHT, other example set-partitioning functions use only the higher-order bits of the PC, or divide based on opcode. This type of history is called *per-set* branch history, and the table is called a *per-set branch history table* (SBHT). Yeh and Patt use the letters G (for global), P (for per-address) and S (for per-set) to denote the different variations of the two-level branch prediction algorithm.

The choice of nonhistory bits used in the PHT index provide for several additional variations. The first option is to simply ignore the PC and use only the BHR to index into the PHT. All branches thus share the entries of the PHT, and this is

called a *global pattern history table* (gPHT), which is used in the example of Figure 9.7. The second alternative, already illustrated in Figure 9.6, is to use the lower bits of the PC to create a *per-address pattern history table* (pPHT). The last variation is to apply some other hashing function (analogous to the hashing function for the per-set BHT) to provide the nonhistory index bits for a *per-set pattern history table* (sPHT).

Yeh and Patt use the letters g, p, and s to indicate these three indexing variations. Combined with the three branch history options (G, P, and S), there are a total of nine variations of two-level predictors using this taxonomy. The notation presented by Yeh and Patt is of the form xAy , where x is G, P, or S, and y is g, p, or s. Therefore, the nine two-level predictors are GAg, GAp, GAs, PAg, PAp, PAs, SAg, SAp, and SAs. In general, the two-level predictors identify patterns of branch outcomes and associate a prediction with each pattern. This captures correlations with complex branch patterns that the simpler Smith predictors cannot track.

9.3.1.3 Index-Sharing Predictors. The two-level algorithm requires the branch predictor designer to make a tradeoff between the width of the BHR (the number of history bits to use) and the number of branch address bits used to index the PHT. For a fixed PHT size, employing a larger number of history bits reveals more opportunities to correlate with more distant branches, but this comes at the cost of using fewer branch address bits. For example, consider again our car navigation analogy. Assume that our driver has a limited memory and can only remember a sequence of at most six letters. She could choose to remember the first five letters of the street name and the one most recent turn decision. This allows her to distinguish between many street names, but has very little decision history information to correlate against. Alternatively, she could choose to only remember the first two letters of the street name, while recording the four most recent turn decisions. This provides more decision history, but she may get confused between *Broad Street* and *Bridge Street*.

Note that if the history length is long, the frequently occurring history patterns will map into the PHT in a very sparse distribution. For example, consider the local history pattern used in Figure 9.7. Since the history length is 4 bits long, there are 16 entries in the PHT. But for the particular pattern used in the example, only 4 of the 16 entries are ever accessed. This indicates that the index formation for the two-level predictor introduces inefficiencies.

McFarling [1993] proposed a variation of a global-history two-level predictor called *gshare*. The *gshare* algorithm attempts to make better use of the index bits by hashing the BHR and the PC *together* to select an entry from the PHT. The hashing function used is a bit-wise exclusive-OR operation. The combination of the BHR and PC tends to contain more information due to the nonuniform distribution of PC values and branch histories. This is called *index sharing*.

Figure 9.9 illustrates a set of PC and branch history pairs and the resulting PHT indices used by the GAp and *gshare* algorithms. Because the GAp algorithm is forced to trade off the number of bits used between the BHR width and the PC bits used, some information from one of these two sources must be left out. In the

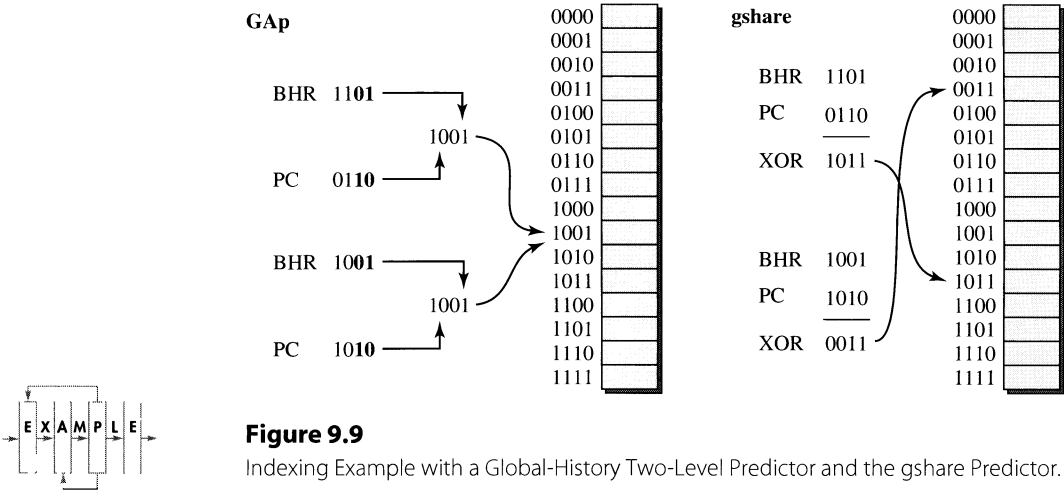


Figure 9.9 Indexing Example with a Global-History Two-Level Predictor and the gshare Predictor.

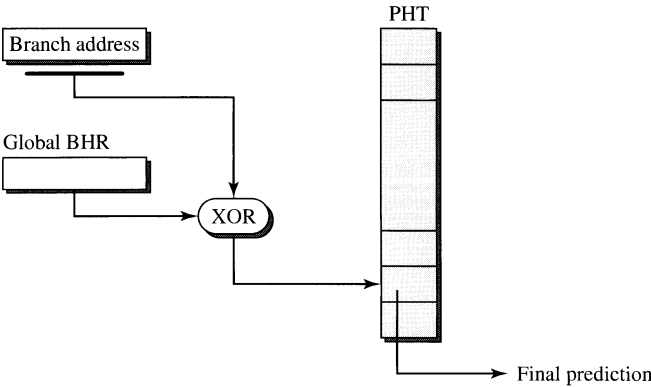


Figure 9.10 The gshare Predictor.

example, the GAP algorithm uses 2 bits from the PC and 2 bits from the global history. Notice that even though the overall PC and history bits are different, using only 2 bits from each causes the two to both map to entry 1001. On the other hand, the exclusive-OR of the 4 bits of the branch address with the full 4 bits of the global history yields different distinct PHT indices.

The hardware for the gshare predictor is shown in Figure 9.10. The circuit is very similar to the global history two-level predictor, except that the concatenation operator for the PHT index has been replaced with an XOR operator. If the number of global history bits used h is less than the number of branch address bits used m , then the global history is XORed with the *upper* h bits of the m branch address bits. The reason for this is that the upper bits of the PC tend to be sparser than the lower-order bits.

Evers et al. [1996] proposed a variation of the gshare predictor that uses a per-address branch history table to store local branch history. The *pshare* algorithm is the local-history analog of the gshare algorithm. The low-order bits of the branch address are used to index into the first-level BHT in the same fashion as the PAg/PAs/PAP two-level predictors. Then the contents of the indexed BHR are XORed with the branch address to form the PHT index.

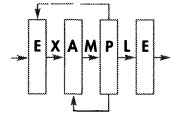
Index sharing predictors are commonly used in modern branch predictors. For example, the IBM Power4 microprocessor's global-history predictor uses an 11-bit global history (BHR) and a 16,384-entry PHT [Tendler et al., 2002]. The Alpha 21264 also makes use of a global history predictor with a 12-bit global history and a 4096-entry PHT [Kessler, 1999]. The amount of available storage, commonly measured in bytes of state, is often the deciding factor in how many PHT entries to use. More recently with the steady increase in clock frequencies, the latency of the PHT access is also becoming a limiting factor in the size of the PHT.

The Power4's 16,384-entry PHT requires 2K bytes of storage since each entry is a 1-bit (1/8 byte) counter. The number of bits of history to use is limited by the PHT size and may also depend on the target set of applications. If the most frequently executed programs exhibit behavior that requires a longer branch history to capture, then it is likely that a longer BHR will be employed. The exact behavior of branches will depend on the compiler, the instruction set, and the input to the program, thus making it difficult to choose an optimal history length that performs well across a wide range of applications.

9.3.1.4 Reasons for Mispredictions. Branch mispredictions can occur for a variety of reasons. Some branches are simply hard to predict. Other mispredictions are due to the fact that any realistic branch predictor is limited in size and complexity.

There are several cases where a branch is fundamentally unpredictable. The first time the predictor encounters a branch, it has no past information about how the branch behaves, and so at best the predictor could make a random choice and expect a 50% prediction rate. With predictors that use branch histories, a similar situation occurs any time the predictor encounters a new branch history pattern. A predictor needs to see a particular branch (or branch history) a few times before it learns the proper prediction that corresponds to the branch (or branch history). During this *training* period, it is unlikely that the predictor will perform very well. For a branch history of length n , there are 2^n possible branch history patterns, and so the training time for a predictor increases with the history length. If the program enters a new phase of execution (for example, a compiler going from parsing to type-checking), branch behaviors may change and the predictor must relearn the new patterns.

Another case where branches are unpredictable is when the data involved in the program are intrinsically random. For example, a program that processes compressed data may have many hard-to-predict branches because well-compressed input data will appear to be random. Other application areas that may have hard-to-predict branches include cryptography and randomized algorithms.



The physical constraints on the size of branch predictors introduces additional sources of branch mispredictions. For example, if a branch predictor has a 128-entry table of counters, and there are 129 distinct branches in a program, then there will be at least one entry that has two different branches mapped to it. If one of these branches is always taken and the other is always not taken, then they will *interfere* with each other and cause branch mispredictions. Such interference is called *negative interference*. If both of these branches are always taken (or both are always not taken), they would still interfere, but no additional mispredictions would be generated; this is called *neutral interference*. Interference is also called *aliasing* because both branches are aliases for the same predictor entry.

Aliasing can occur even if there are more predictor entries than branches. With a 128-entry table, let us assume that the hashing function is the remainder of the branch address when divided by 128 (i.e., $\text{index} := \text{address} \bmod 128$). There may only be two branches in the entire program, but if their addresses are 131 and 259, then both branches will still map to predictor entry 3. This is called *conflict aliasing*. This is similar to the case in our car driving analogy where our driver gets confused by Broad St. and Bridge St. because she is only remembering the first two letters and they happen to be the same.

Some branches are predictable, but a particular predictor may still mispredict the branch because the predictor does not have the right information. For example, consider a branch that is strongly correlated with the ninth-most recent branch. If a predictor only uses an 8-bit branch history, then the predictor will not be able to accurately make this prediction. Similarly, if a branch is strongly correlated to a previous local history bit, then it will be difficult for a global history predictor to make the right prediction.

More sophisticated prediction algorithms can deal with some classes or types of mispredictions. For capacity problems, the only solution is to increase the size of the predictor structures. This is not always possible due to die area, latency, and/or power constraints. For conflict aliasing, a wide variety of algorithms have been developed to address this problem, and many of these are described in Section 9.3.2. Furthermore, many algorithms have been developed to make use of different types of information (such as global vs. local branch histories or short vs. long histories), and these are covered in Section 9.3.3.

9.3.2 Interference-Reducing Predictors

The PHT used in the two-level and gshare predictors is a direct-mapped, tagless structure. Aliasing occurs between different address-history pairs in the PHT. The PHT can be viewed as a cache-like structure, and the three-C's model of cache misses [Hill, 1987; Sugumar and Abraham, 1993] gives rise to an analogous model for PHT aliasing [Michaud et al., 1997]. A particular address-history pair can “miss” in the PHT for the following reasons:

1. *Compulsory aliasing* occurs the first time the address-history pair is ever used to index the PHT. The only recourse for compulsory aliasing is to initialize the PHT counters in such a way that the majority of such lookups

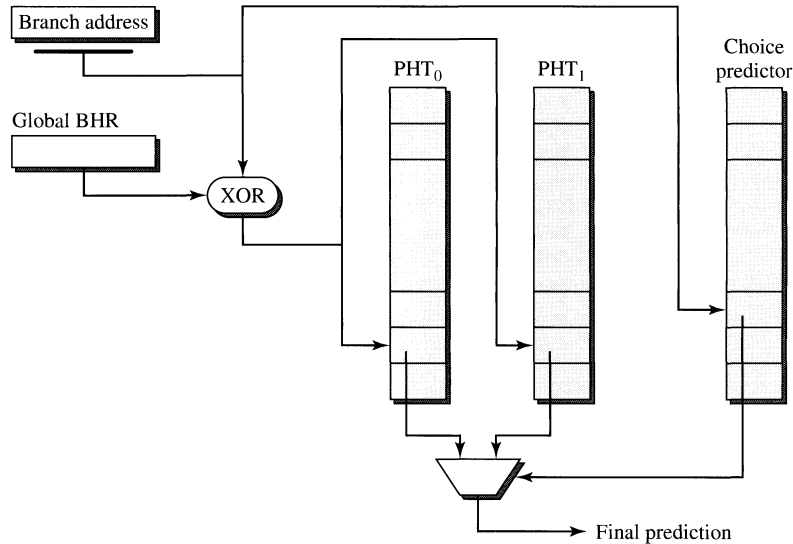
still yield accurate predictions. Fortunately, Michaud et al. show that compulsory aliasing accounts for a very small fraction of all branch prediction lookups (much less than 1% on the IBS benchmarks [Richard Uhlig et al., 1995]).

2. *Capacity aliasing* occurs because the size of the current working set of address-history pairs is greater than the capacity of the PHT. This aliasing can be mitigated by increasing the PHT size.
3. *Conflict aliasing* occurs when two different address-history pairs map to the same PHT entry. Increasing the PHT size often has little effect on reducing conflict aliasing. For caches, the associativity can be increased or a better replacement policy can be used to reduce the effects of conflicts.

For caches, the standard solution for conflict aliasing is to increase the associativity of the cache. Even for a direct-mapped cache, address tags are necessary to determine whether the cached item belongs to the requested address. Branch predictors are different because tags are not required for proper operation. In many cases, there are other ways to use the available transistor budget to deal with conflict aliasing than the use of associativity. For example, instead of adding a 2-bit tag to every saturating 2-bit counter, the size of the predictor could instead be doubled. Sections 9.3.2.1 to 9.3.2.6 describe a variety of ways to deal with the problem of interference in branch predictors. These predictors are all global-history predictors because global-history predictors are usually more accurate than local-history predictors, but the ideas are equally applicable to local-history predictors as well. Note that many of these algorithms are often referred to as *two-level* branch predictors, since they all use a first level of branch history and a second level of counters or other state that provides the final prediction.

9.3.2.1 The Bi-Mode Predictor. The Bi-Mode predictor uses multiple PHTs to reduce the effects of aliasing [Lee et al., 1997]. The Bi-Mode predictor consists of two PHTs (PHT_0 and PHT_1), both indexed in a gshare fashion. The indices used on the PHTs are identical. A separate *choice predictor* is indexed with the lower-order bits of the branch address only. The choice predictor is a table of 2-bit counters (identical to a Smith₂ predictor), where the most-significant bit indicates which of the two PHTs to use. In this manner, the branches that have a strong taken bias are placed in one PHT and the branches that have a not-taken bias are separated into the other PHT, thus reducing the amount of destructive interference. The two PHTs have identical sizes, although the choice predictor may have a different number of entries.

Figure 9.11 illustrates the hardware for the Bi-Mode predictor. The branch address and global branch history are hashed together to form an index into the PHTs. The same index is used on both PHTs, and the corresponding predictions are read. Simultaneously, the low-order bits of the branch address are used to index the choice predictor table. The prediction from the choice predictor drives the select line of a multiplexer to choose one of the two PHT banks.

**Figure 9.11**

The Bi-Mode Predictor.

The rationale behind the Bi-Mode predictor is that most branches are biased toward one direction or the other. The choice predictor effectively remembers what the bias of each branch is. Branches that are more strongly biased toward one direction all use the same PHT. The result is that even if two branches map to the same entry in this PHT, they are more likely to go in the same direction. The result is that an opportunity for negative interference has been converted to neutral interference.

The PHT bank selected by the choice predictor is always updated when the final branch outcome has been determined. The other PHT bank is not updated. The choice predictor is always updated with the branch outcome, except in the case where the choice predictor's direction is the opposite of the branch outcome, but the overall prediction of the selected PHT bank was correct. These update rules implement a *partial update* policy.

9.3.2.2 The gskewed Predictor. The gskewed algorithm divides the PHT into three (or more) banks. Each bank is indexed by a different hash of the address-history pair. The results of these three lookups are combined by a majority vote to determine the overall prediction. The intuition is that if the hashing functions are different, even if two address-history pairs destructively alias to the same PHT entry in one bank, they are unlikely to conflict in the other two banks. The hashing functions f_0 , f_1 , and f_2 presented in Michaud et al. [1997] have the property that if $f_0(x_1) = f_0(x_2)$, then $f_1(x_1) \neq f_1(x_2)$ and $f_2(x_1) \neq f_2(x_2)$ if $x_1 \neq x_2$. That is, if two addresses conflict in one PHT, they are guaranteed to not conflict with each other in

the other two PHTs. For three banks of 2^n -entry PHTs, the definitions of the three hashing functions are

$$f_0(x, y) = H(y) \oplus H^{-1}(x) \oplus x \quad (9.1)$$

$$f_1(x, y) = H(y) \oplus H^{-1}(x) \oplus y \quad (9.2)$$

$$f_2(x, y) = H^{-1}(y) \oplus H(x) \oplus x \quad (9.3)$$

where $H(b_n, b_{n-1}, \dots, b_3, b_2, b_1) = (b_n \oplus b_1, b_n, b_{n-1}, \dots, b_3, b_2)$, H^{-1} is the inverse of H , and x and y are each n bits long. For the gskewed algorithm, the arguments x and y of the hashing functions are the n low-order bits of the branch address, and the n most recent global branch outcomes, respectively.

The amount of conflict aliasing is a result of the hashing function used to map the PC-history pair into a PHT index. Although the gshare exclusive-OR hash can remove certain types of interference, it can also introduce interference as well. Two different PC values and two different histories can still result in the same index. For example, the PC-history pairs $(PC = 0110) \oplus (\text{history} = 1100) = 1010$, and $1101 \oplus 0111 = 1010$ map to the same index.

The hardware for the gskewed predictor is illustrated in Figure 9.12. The branch address and the global branch history are hashed separately with the three hashing functions (9.1) through (9.3). Each of the three resulting indices is used to address a different PHT bank. The direction bits from the 2-bit counters in the PHTs are combined with a majority function to make the final prediction.

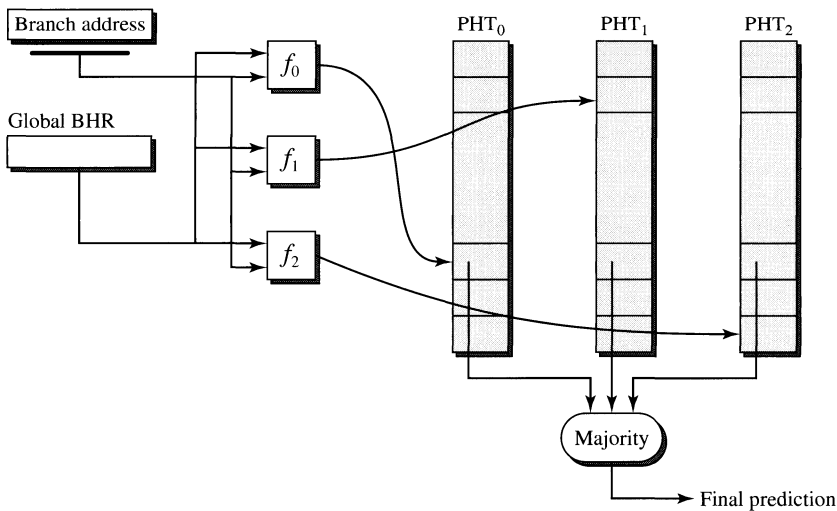
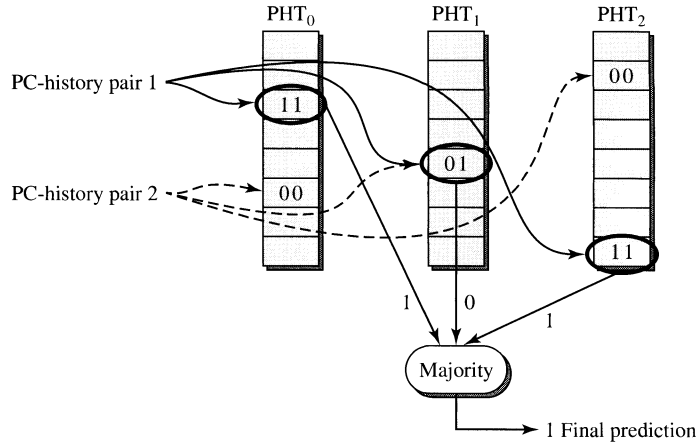
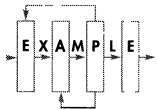


Figure 9.12

The gskewed Predictor.

**Figure 9.13**

A gskewed Example Showing How the Majority Function Can Tolerate Interference in One of the PHT Banks.

Figure 9.13 shows a gskewed predictor example with two sets of PC-history pairs corresponding to two different branches. In this example, one PC-history pair corresponds to a strongly taken branch, whereas the other PC-history pair corresponds to a strongly not-taken branch. The two branches map to the same entry in PHT_1 which causes destructive interference. The hashing functions (9.1) through (9.3) guarantee that a conflict in one PHT means there are no conflicts between these two branches in both of the other two PHTs. As a result, the majority function can effectively mask the one disagreeing vote and still provide the correct prediction.

Two different update policies for the gskewed algorithm are *total update* and *partial update*. The total update policy treats each of the PHT banks identically and updates all banks with the branch outcome. The partial update policy does not update a bank if that particular bank mispredicted, but the overall prediction was correct. The partial update policy improves the overall prediction rate of the gskewed algorithm. When only one of the three banks mispredicts, it is not updated, thus allowing it to contribute to the correct prediction of another address-history pair.

The choice of the branch history length involves a tradeoff between capacity and aliasing conflicts. Shorter branch histories tend to reduce the amount of possible aliasing because there are fewer possible address-branch history pairs. On the other hand, longer histories tend to provide better branch prediction accuracy because there is more correlation information available. A modification to the gskewed predictor is the *enhanced gskewed predictor*. In this variation, PHT banks 1 and 2 are indexed in the usual fashion using the branch address, global history, and the hashing functions f_1 and f_2 , while PHT bank 0 is indexed only by the lower bits of the program counter. The rationale behind this approach is as follows. When the history length becomes larger, the number of branches between one instance of a branch address-branch history pair, and another identical instance tends to increase. This increases the

probability that aliasing will occur in the meantime and corrupt one of the banks. Since the first bank of the enhanced gskewed predictor is addressed by the branch address only, the distance between successive accesses will be shorter, and so the likelihood that an unrelated branch aliases to the same entry in PHT_0 is decreased.

A variant of the enhanced gskewed algorithm was selected to be used in the Alpha EV8 microprocessor [Seznec et al., 2002], although the EV8 project was eventually cancelled in a late phase of development.

9.3.2.3 The Agree Predictor. The gskewed algorithm attempts to reduce the effects of conflict aliasing by storing the branch prediction in multiple locations. The agree predictor reduces destructive aliasing interference by reinterpreting the PHT counters as a *direction agreement* bit [Spangle et al., 1997].

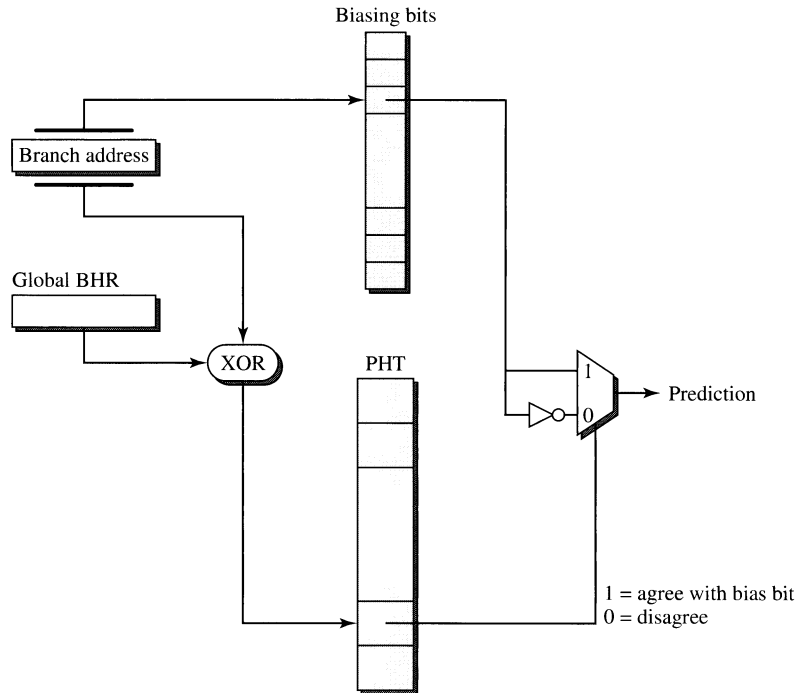
When two address-history pairs map into the same PHT entry, there are two types of interference that can result. The first is *destructive* or *negative* interference. Destructive interference occurs when the counter updates of one address-history pair corrupt the stored state of a different address-history pair, thus causing more mispredictions. The address-history pairs that result in destructive interference are each trying to update the counter in opposite directions; that is, one address-history pair is consistently incrementing the counter, and the other pair attempts to decrement the counter. The other type of interference is *neutral* interference where the PHT entry correctly predicts the branch outcomes for both address-history pairs.

Regardless of the actual direction of the history-address pairs, branches tend to be heavily biased in one direction or the other. In other words, in an infinite-entry PHT where there is no interference, the majority of counters will be either in the strongly taken (ST) or strongly not-taken (SN) states.

The agree predictor stores the most likely predicted direction in a separate *biasing bit*. This biasing bit may be stored with the branch target buffer (see Section 9.5.1.1) line of the corresponding branch, or in some other separate hardware structure. The biasing bit may be initialized to the outcome of the first instance of the branch, or it may be a branch hint inserted by the compiler. Instead of predicting the branch direction, the PHT counter now predicts whether or not the branch will go in the same direction as the corresponding biasing bit. Another interpretation is that the PHT counter predicts whether the branch outcome will *agree* with the biasing bit.

Figure 9.14 illustrates the hardware for the agree predictor. Like the gshare algorithm, the branch address and global branch history are combined to index into the PHT. At the same time, the branch address is also used to look up the biasing bit. If the most-significant bit of the indexed PHT counter is a one (predict agreement with the biasing bit), then the final branch prediction is equal to the biasing bit. If the most significant bit is a zero (predict disagreement with the biasing bit), then the complement of the biasing bit is used for the final prediction. The number of biasing bits stored is generally different than the number of PHT entries.

After a branch instruction has resolved, the corresponding PHT counter is updated based on whether or not the actual branch outcome agreed with the biasing bit. In this fashion, two different address-history pairs may conflict and map to the same PHT entry, but if their corresponding biasing bits are set accurately, the predictions will not

**Figure 9.14**

The Agree Predictor.

be affected. The agree prediction mechanism is used in the HP PA-RISC 8700 processor [Hewlett Packard Corporation, 2000]. Their biasing bits are determined by a combination of compiler analysis of the source code and profile-based optimization.

9.3.2.4 The YAGS Predictor. The Bi-Mode predictor study demonstrated that the separation of branches into two separate mostly taken and mostly not-taken substreams is beneficial. The yet another global scheme (YAGS) approach is similar to the Bi-Mode predictor, except that the two PHTs record only the instances that do not agree with the direction bias [Eden and Mudge, 1998]. The PHTs are replaced with a T-cache and an NT-cache. Each cache entry contains a 2-bit counter and a small tag (6 to 8 bits) to record the branch instances that do not agree with their overall bias. If a branch does not have an entry in the cache, then the selection counter is used to make the prediction. The hardware is illustrated in Figure 9.15.

To make a branch prediction with the YAGS predictor, the branch address indexes a *choice PHT* (analogous to the choice predictor of the Bi-Mode predictor). The 2-bit counter from the choice PHT indicates the bias of the branch *and* is used to select one of the two caches. If the choice PHT counter indicates taken, then the NT-Cache is consulted. The NT-Cache is indexed with a hash of the branch address and the global history, and the stored tag is compared to the least-significant bits of the

substreams. The insight for the YAGS predictor is that the PHT counter values in the second-level PHTs of the Bi-Mode predictor are mostly redundant with the information conveyed by the choice predictor, and so it only allocates hardware resources to make note of the cases where the prediction does not match the bias.

In the YAGS study, two-way associativity was also added to the T-cache and NT-cache, which only required the addition of 1 bit to maintain the LRU state. The tags that are already stored are reused for the purposes of associativity, and only an extra comparator and simple logic need to be added. The replacement policy is LRU, with the exception that if the counter of an entry in the T-cache indicates not-taken, it is evicted first because this information is already captured by the choice PHT. The reverse rule applies for entries in the NT-cache. The addition of two-way associativity slightly increases prediction accuracy, although it adds some additional hardware complexity as well.

9.3.2.5 Branch Filtering. Branches tend to be highly biased toward one direction or the other, and the Bi-Mode algorithm works well because it sorts the branches based on their bias which reduces negative interference. A different approach called *branch filtering* attempts to remove the highly biased branches from the PHT, thus reducing the total number of branches stored in the PHT which helps to alleviate capacity and conflict aliasing [Change et al., 1996]. The idea is to keep track of how many times each branch has gone in the same direction. If a branch has taken the same direction more than a certain number of times, then it is “filtered” in that it will no longer make updates to the PHT.

Figure 9.16 shows the organization of the branch counting table and the PHT, along with the logic for detecting whether a branch should be filtered. Although this figure shows branch filtering with a gshare predictor, the branch filtering technique could be applied to other prediction algorithms as well. An entry in the branch counting table tracks the branch direction, and how many consecutive times the branch has taken that direction. If the direction changes, the new direction is stored and the count is reset. If the counter has been incremented to its maximum value, then the corresponding branch is deemed to be very highly biased. At this point, this branch will no longer update the PHT, and the branch count table provides the prediction. If at any point the direction changes for this branch, then the counter is reset and the PHT once again takes over making the predictions.

Branch filtering effectively removes branches corresponding to error-checking code, such as the almost-never-taken malloc checking branch in the example from Section 9.2.3, and other dynamically constant branches. Although the branch counting table has been described here as a separate entity, the counter and direction bit would actually be part of the branch target buffer, which is described in Section 9.5.1.1.

9.3.2.6 Selective Branch Inversion. The previous several branch prediction schemes all aim to provide better branch prediction rates by reducing the amount of interference in the PHT (interference avoidance). Another approach, *selective branch inversion* (SBI), attacks the interference problem differently by using interference

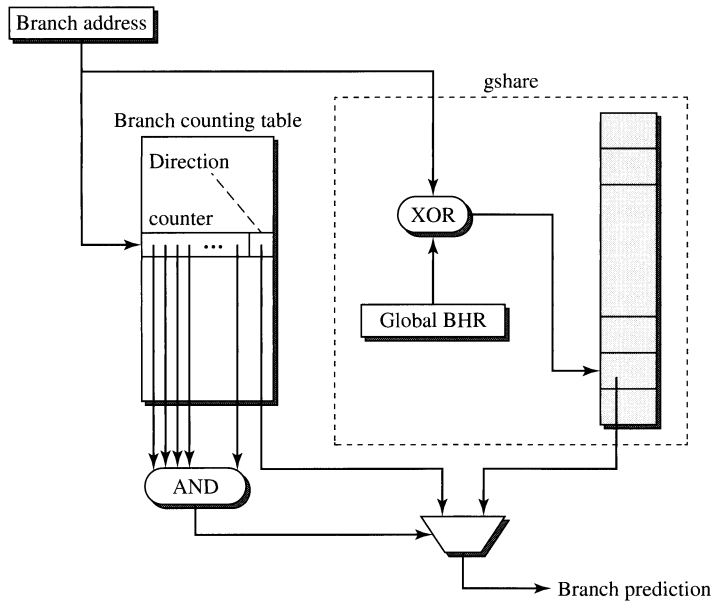


Figure 9.16
Branch Filtering Applied to a gshare Predictor.

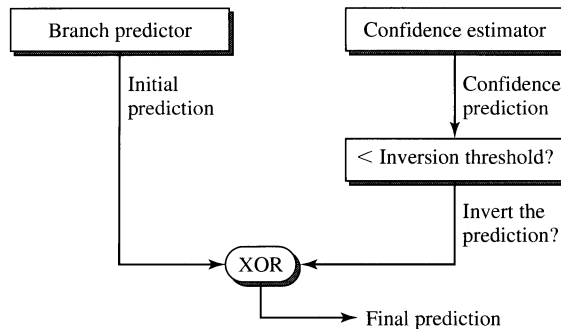


Figure 9.17
Selective Branch Inversion Applied to a Generic Branch Predictor.

correction [Argon et al., 2001; Manne et al., 1999]. The idea is to estimate the *confidence* of each branch prediction; if the confidence is lower than some threshold, then the direction of the branch prediction is inverted. See Section 9.5.2 for an explanation of predicting branch confidence. A generic SBI predictor is shown in Figure 9.17. Note that the SBI technique can be applied to any existing branch prediction scheme. An SBI gskewed or SBI Bi-Mode predictor achieves better prediction rates by performing both interference avoidance and interference correction.

9.3.3 Predicting with Alternative Contexts

Correlating branch predictors are basically simple pattern-recognition mechanisms. The predictors learn mappings from a *context* to a branch outcome. That is, every time a branch outcome becomes known, the predictor makes note of the current context. In the future, should the same context arise, the predictor will make a prediction that corresponds to the previous time(s) it encountered that context. So far, the predictors described in this chapter have used some combination of the branch address and the branch outcome history as the context for making predictions.

There are many design decisions that go into choosing the context for a branch predictor. Should the predictor use global history or local history? How many of the most recent branch outcomes should be used? How many bits of the branch address should be included? How is all of this information combined to form the final context?

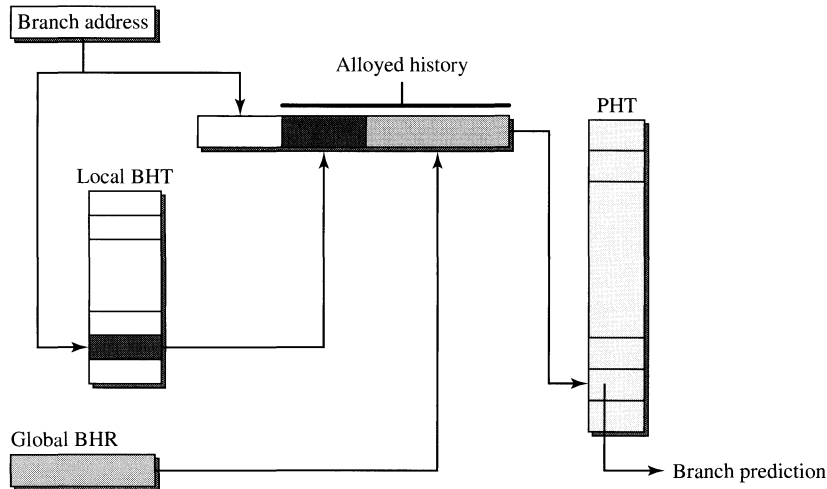
In general, the more context a predictor uses, the more opportunities it has for detecting correlations. Using the same example given in Section 9.3.1.4, a branch correlated to a branch outcome nine branches ago will not be accurately predicted by a predictor that makes use of a history that is only eight branches deep. That is, an eight-deep branch history does not provide the proper context for making this prediction.

The predictors described here all improve prediction accuracies by making use of better context. Some use a greater amount of context, some use different contexts for different branches, and some use additional types of information beyond the branch address and the branch history.

9.3.3.1 Alloyed History Predictors. The GA* predictors are able to make predictions based on correlations with the global branch history. The PA* predictors use correlations with local, or per-address, branch history. Programs may contain some branches whose outcomes are well predicted by global-history predictors *and* other branches that are well predicted by local-history predictors. On the other hand, some branches require *both* global branch history and per-address branch history to be correctly predicted. Mispredictions due to using the wrong type of history or only one type when more than one are needed are called *wrong-history mispredictions*.

An alloyed branch predictor removes some of these wrong-history mispredictions by using *both* global and local branch history [Skadron et al., 2003]. A per-address BHT is maintained as well as a global branch history register. Bits from the branch address, the global branch history, and the local branch history are all concatenated together to form an index into the PHT. The combined global/local branch history is called *alloyed* branch history. This approach allows both global and local correlations to be distinguished by the same structure. Alloyed branch history also enables the branch predictor to detect correlations that simultaneously depend on both types of history; this class of predictions is one that could not be successfully predicted by either a global-history predictor or a local-history predictor alone.

Alloyed predictors can also be classified as MAg/MAs/MAP predictors (M for “merged” history), where the second-level table can be indexed in the same way as the two-level predictors. Therefore, the three basic alloyed predictors are MAg, MAP, and

**Figure 9.18**

The Alloyed History Predictor

MAAs. Alloyed history versions of other branch prediction algorithms are also possible, such as mshare (alloyed history gshare), or mskewed (alloyed history gskewed).

Figure 9.18 illustrates the hardware organization for the alloyed predictor. Like the PAg/PAs/PAP two-level predictors, the low-order bits of the branch address are used to index into the local history BHT. The corresponding local history is then concatenated with the contents of the global BHR and the bits from the branch address. This index is used to perform a lookup in the PHT, and the corresponding counter is used to make the final branch prediction. The branch predictor designer must make a tradeoff between the width of the global BHR and the width of the per-address BHT entries.

9.3.3.2 Path History Predictors. With the outcome history-based approaches to branch prediction, it may be the case that two very different paths of the program execution may have overlapping branch address and branch history pairs. For example, in Figure 9.19, the program may reach branch X in block D by going through blocks A, C, and D, or going through B, C, and D. When attempting to predict branch X in block D, the branch address and the branch histories for the last two global branches are identical for either ACD or BCD. Depending on the *path* by which the program arrived at block D, branch X is primarily not-taken (for path ACD), or primarily taken (for path BCD). When using only the branch outcome history, the different branch outcome patterns will cause a great deal of interference in the corresponding PHT counter.

Path-based branch correlation has been proposed to make better branch predictions when dealing with situations like the example in Figure 9.19. Instead of storing the last n branch outcomes, k bits from each of the last n branch addresses

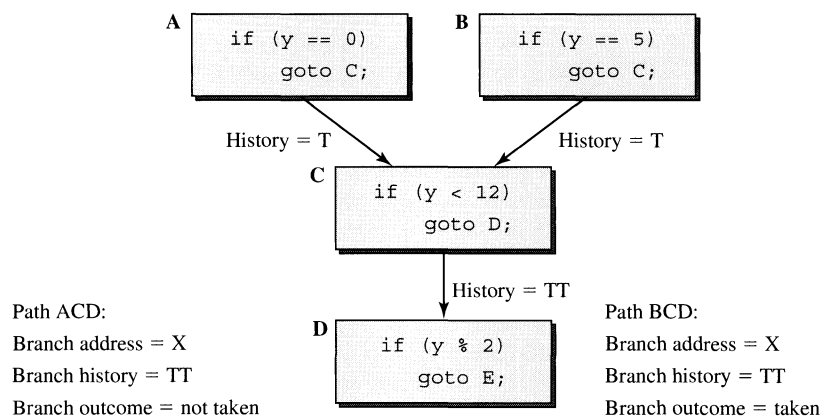


Figure 9.19
Path History Example.

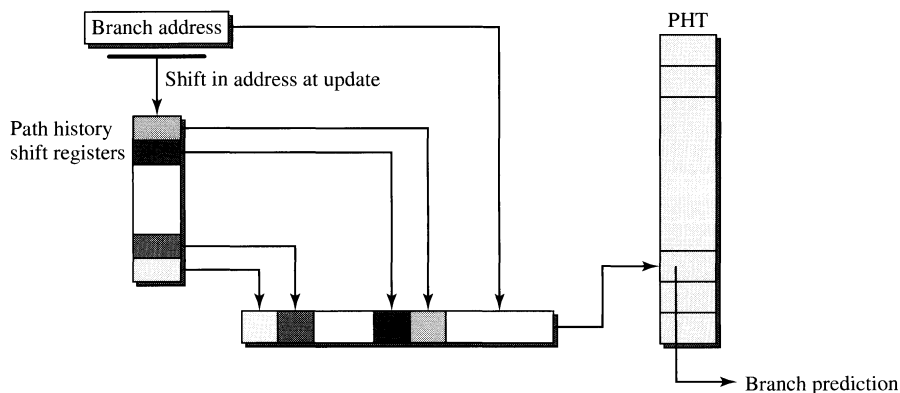


Figure 9.20
A Path History Predictor.

are stored [Nair, 1995; Reches and Weiss, 1997]. The concatenation of these nk bits encodes the branch path of the last n branches, also called the *path history*, thus potentially allowing the predictor to differentiate between the two very different branch behaviors in the example of Figure 9.19. Combined with a subset of the branch address bits of the current branch, this forms an index into a PHT. The prediction is then made in the same way as a normal two-level predictor.

Figure 9.20 illustrates the hardware for the path history branch predictor. The bits from the last n branches are concatenated together to form a path history. The path history is then concatenated with the low-order bits of the current branch

address. This index is used to perform a lookup in the PHT, and the final prediction is made. After the branch is processed, bits from the current branch address are added to the path history, and the oldest bits are discarded. The path history register can be implemented with shift registers. The number of bits per branch address to be stored k , the number of branches in the path history n , and the number of bits from the current branch address m all must be carefully chosen. The PHT has 2^{nk+m} entries, and therefore the area requirements can become prohibitive for even moderate values of n , k , and m . Instead of concatenating the n branch addresses, combinations of shifting, rotating, and hashing (typically using XORs) can be used to compress the $nk + m$ bits down to a more manageable size [Stark et al., 1998].

9.3.3.3 Variable Path Length Predictors. Some branches are correlated to branch outcomes or branch addresses that occurred very recently. Incorporating a longer history introduces additional bits that do not provide any additional information to the predictor. In fact, this useless context can degrade the performance of the predictor because the predictor must figure out what parts of the context are irrelevant, which in turn increases the training time. On the other hand, some branches are strongly correlated to older branches, which requires the predictor to make use of a longer history if these branches are to be correctly predicted.

One approach to dealing with the varying history length requirements of branches is to use different history lengths for each branch [Stark et al., 1998]. The following description uses path history, but the idea of using different history lengths can be applied to branch outcome histories as well. Using n different hashing functions $f_1, f_2, \dots, f_n$, hash function f_i creates a hash of the last i branch addresses in the path history. The hash function used may be different between different branches, thus allowing for variable-length path histories. The selection of which hash function to use can be determined statically by the compiler, chosen with the aid of program profiling, or dynamically selected with additional hardware for tracking how well each of the hash functions is performing.

The elastic history buffer (EHB) uses a variable outcome history length [Tarlescu et al., 1996]. A profiling phase statically chooses a branch history length for each static branch. The compiler communicates the chosen length by using branch hints.

9.3.3.4 Dynamic History Length Fitting Predictors. The optimal history length to use in a predictor varies between applications. Some applications may have program behaviors that change frequently and are better predicted by more adaptive short-history predictors because short-history predictors require less time to train. Other programs may have distantly correlated branches, which require long histories to detect the patterns. By fixing the branch history length to some constant, some applications may be better predicted at the cost of reduced performance for others. Furthermore, the optimal history length for a program may change during the execution of the program itself. Any multiphased computation such as a compiler may exhibit very different branch patterns in the different phases of execution. A short history may be optimal in one phase, and a longer history may provide better prediction accuracy in the next phase.

Dynamic history length fitting (DHLF) addresses the problem of varying optimal history lengths. Instead of fixing the history length to some constant, the predictor uses different history lengths and attempts to find the length that minimizes branch mispredictions [Juan et al., 1998]. For applications that require shorter histories, a DHLF predictor will tune itself to consider fewer branch outcomes; for benchmarks that require longer histories, a DHLF predictor will adjust for that situation as well. The DHLF technique can be applied to all kinds of correlating predictors (gshare, Bi-Mode, gskewed, etc.).

9.3.3.5 Loop Counting Predictors. In general, the termination of a for-loop is difficult to predict using any of the algorithms already presented in this section. Each time a for-loop is encountered, the number of iterations executed is often the same as the previous time the loop was encountered. A simple example of this is the inner loop of a matrix multiply algorithm where the number of iterations is equal to the matrix block size. Because of the consistent number of iterations, the loop exit branch should be very easy to predict. Unfortunately, a branch history register-based approach would require BHR sizes greater than the number of iterations of the loop. Beyond a small number of iterations, the storage requirements for such a predictor become prohibitive, because the PHT size is exponential in the history length.

The Pentium-M processor uses a loop predictor in conjunction with a branch history-based predictor [Gochman et al., 2003]. The loop predictor consists of a table where each entry contains fields to record the current iteration count, the iteration limit, and the direction of the branch. This is illustrated in Figure 9.21. A *loop* branch is one that always goes the same direction (either taken or not-taken) followed by a single instance where the branch direction is the opposite, and then this pattern repeats. A traditional loop-closing branch has a pattern of 111...1110111...1110111..., but the Pentium-M loop predictor can also handle the

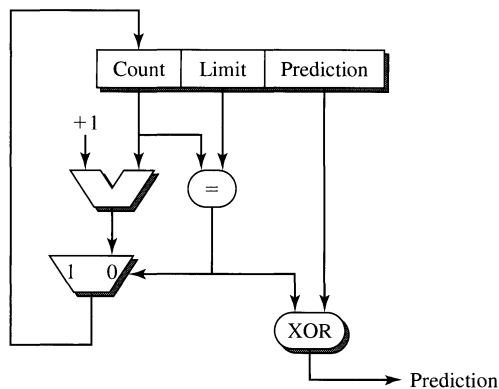


Figure 9.21

A Single Entry of the Loop Predictor Table Used in the Pentium-M Processor.

opposite pattern of 000 . . . 0001000 . . . 0001000 The limit field stores a count of the number of iterations that were observed for the previous invocation of the loop. When a loop exit is detected, the counter value is copied to the limit field and then the counter is reset for the next run of the loop. The prediction field records the predominant direction of the branch. As long as the counter is less than the limit, the loop predictor will use the prediction field. When the counter reaches the limit, this indicates that the predictor has reached the end of the loop, and so it predicts in the opposite direction as that stored in the prediction field.

While loop-counting predictors are useful in hybrid predictors (see Section 9.4), they provide very poor performance when used by themselves because they cannot capture nonloop behaviors.

9.3.3.6 The Perceptron Predictor. By maintaining larger branch history registers, the additional history stored provides more opportunities for correlating the branch predictions. There are two major drawbacks with this approach. The first is that the size of the PHT is exponential in the width of the BHR. The second is that many of the history bits may not actually be relevant, and thus act as training “noise.” Two-level predictors with large BHR widths take longer to train.

One solution to this problem is the Perceptron predictor [Jimenez and Lin, 2003]. Each branch address (*not* address-history pair) is mapped to a single entry in a Perceptron table. Each entry in the table consists of the state of a single Perceptron. A Perceptron is the simplest form of a neural network [Rosenblatt, 1962]. A Perceptron can be trained to learn certain boolean functions.

In the case of the Perceptron branch predictor, each bit x_i of the input x is equal to 1 if the branch was taken ($BHR_i = 1$) and x_i is equal to -1 if the branch was not taken ($BHR_i = 0$). There is one special *bias* input x_0 which is always 1. The Perceptron has one weight w_i for each input x_i , including one weight w_0 for the bias input. The Perceptron’s output y is computed as

$$y = w_0 + \sum_{i=1}^n (w_i \cdot x_i)$$

If y is negative, the branch is predicted to be not taken. Otherwise the branch is predicted to be taken.

After the branch outcome is available, the weights of the Perceptron are updated. Let $t = -1$ if the branch was not taken, and $t = 1$ if the branch was taken. In addition, let $\theta > 0$ be a *training threshold*. The variable y_{out} is computed as

$$y_{\text{out}} = \begin{cases} 1 & \text{if } y > \theta \\ 0 & \text{if } -\theta \leq y \leq \theta \\ -1 & \text{if } y < -\theta \end{cases}$$

Then if y_{out} is not equal to t , all the weights are updated as $w_i = w_i + tx_i$, $i \in \{0, 1, 2, \dots, n\}$. Intuitively, $-\theta \leq y \leq \theta$ indicates that the Perceptron has not been trained to a state where the predictions are made with high confidence. By setting y_{out} to

zero, the condition $y_{\text{out}} \neq t$ will always be true, and the Perceptron's weights will be updated (training continues). When the correlation is large, the magnitude of the weight will tend to become large.

One limitation of using the Perceptron learning algorithm is that only *linearly separable* functions can be learned. Linearly separable boolean functions are those where all instances of outputs that are 1 can be separated in hyperspace from all instances whose outputs are 0 by a hyperplane. In Jimenez and Lin [2003], it is shown that for half of the SPEC2000 integer benchmarks, over 50% of the branches are linearly inseparable. The Perceptron predictor generally performs better than gshare on benchmarks that have more linearly separable branches, whereas gshare outperforms the Perceptron predictor on benchmarks that have a greater number of linearly inseparable branches.

The Perceptron predictor can adjust the weights corresponding to each bit of the history, since the algorithm can effectively “tune out” any history bits that are not relevant (low correlation). Because of this ability to selectively filter the branches, the Perceptron often attains much faster training times than conventional PHT-based approaches.

Figure 9.22 illustrates the hardware organization of the Perceptron predictor. The lower-order bits of the branch address are used to index into the table of

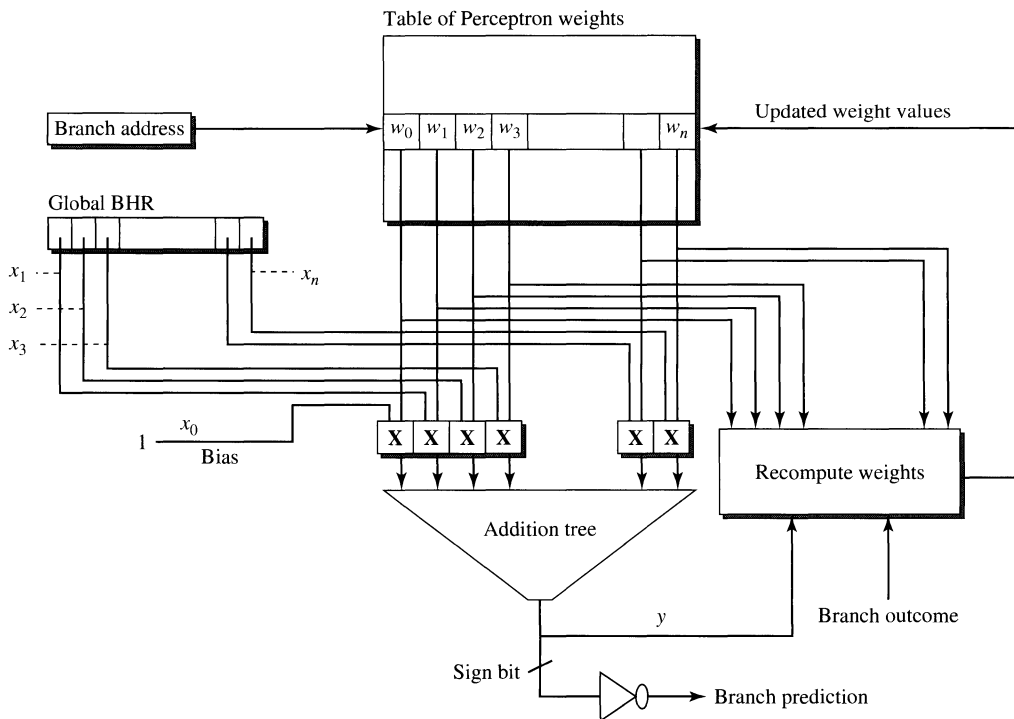


Figure 9.22
The Perceptron Predictor.

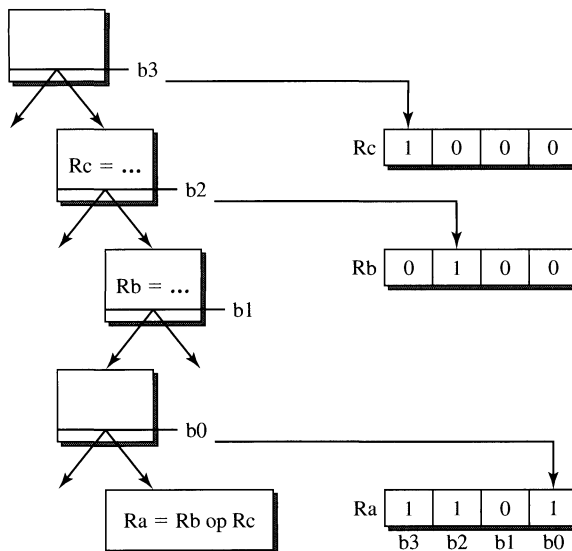
Perceptrons in a per-address fashion. The weights of the selected Perceptron and the BHR are forwarded to a block of combinatorial logic that computes y . The prediction is made based on the complement of the sign bit (most-significant bit) of y . The value of y is also forwarded to an additional block of logic and combined with the actual branch outcome to compute the updated values of the weights of the Perceptron.

The design space for the Perceptron branch predictor appears to be much larger than that of the gshare and Bi-Mode predictors, for example. The Perceptron predictor has four parameters: the number of Perceptrons, the number of bits of history to use, the width of the weights, and the learning threshold. There is an empirically derived relation for the optimal threshold value as a function of the history length. The threshold θ should be equal to $\lfloor 1.93h + 14 \rfloor$, where h is the history length. The number of history bits that can potentially be used is still much larger than in the gshare predictors (and similar schemes).

Similar to the alloyed history two-level branch predictors, alloyed history Perceptron predictors have also been proposed. For n bits of global history and m bits of local history, each Perceptron uses $n + m + 1$ weights (+1 for the bias) to make a branch prediction.

9.3.3.7 The Data Flow Predictor. The Perceptron predictor makes use of a long-history register and effectively finds the highly correlated branches by assigning them higher weights. The majority of these branches are correlated for two reasons. The first is that a branch may guard instructions that affect the test condition of the later branch, such as branch A from the example in Figure 9.4. These branches are called *affector* branches. The second is that the two branches operate on similar data. The Perceptron attempts to find the highly correlated branches in a fuzzy fashion by assigning larger weights to the more correlated branches. Another approach to find the highly correlated branches from a long-branch-history register is the *data flow* branch predictor that explicitly tracks register dependences [Thomas et al., 2003].

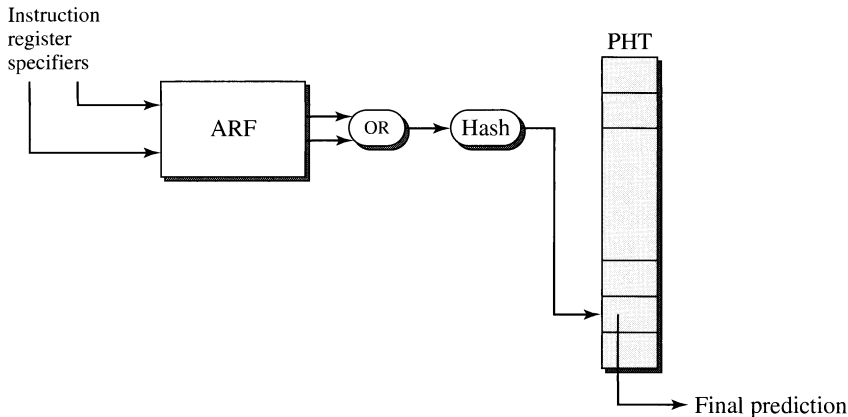
The main idea behind the data flow branch predictor is to explicitly track which previous branches are affector branches for the current branch. The *affector register file* (ARF) stores one bitmask per architected register, where the entries of the bitmask correspond to past branches. If the i th most recent branch is an affector for register R, then the i th most recent bit in entry R of the ARF will be set. For register updating instructions of the form $R_a = R_b \text{ op } R_c$, the ARF entry for R_a is set equal to the bitwise-OR of the ARF entries for R_b and R_c , with the least-significant bit (most recent branch) set to 1. This is illustrated in Figure 9.23. Setting the least-significant bit to one indicates that the most recent branch (b_0) guards an instruction that modifies R_a . Note that the entries for R_b and R_c also have their corresponding affector bits set. The OR of the ARF entries for the operands makes the current register inherit the affectors of its operands. In this fashion, an ARF entry records a bitmask that specifies all the affector branches that can potentially affect the register's value. On a branch instruction, the ARF is updated by shifting all entries to the left by one and filling in the least-significant bit with zero.

**Figure 9.23**

Affector Register File Update for a Register Writing Instruction.

The ARF specifies a set of potentially important branches, because it is these branches that can affect the values of the condition registers. A conditional branch compares one or more register values and evaluates some condition on these values (e.g., equal to zero or greater than). To make a prediction, the data flow predictor uses the ARF entries corresponding to the operand(s) of the current branch; if there are two operands, then a final bitmask is formed by the exclusive-OR of the respective ARF entries. The affector bitmask is ANDed with the global history register, which isolates the global history outcomes for the affector branches only. The branch history register is likely to be larger than the index into the PHT, and so the masked version of the history register still needs to be hashed down to an appropriate size. This final hashed version of the masked history register indexes into a PHT to provide the final prediction. The overall organization of the data flow predictor is illustrated in Figure 9.24.

In the original data flow branch predictor study, the predictor was presented as a *corrector predictor*, which is basically a secondary predictor that backs up some other prediction mechanism. The idea is basically the same as the overriding predictor organization explained in Section 9.5.4.2. A primary predictor provides most of the predictions, and the data flow predictor attempts to learn and provide corrections for the branches that the primary predictor does not properly handle. For this reason, the PHT entries of the data flow predictor may be augmented with partial tags (see partial resolution in Section 9.5.1.1) and set associativity. This allows the data flow predictor to carefully identify and correct only the branches it knows about. Because the data flow predictor only attempts to correct a select set of branches, its total size may be smaller than other conventional stand-alone branch predictors.

**Figure 9.24**

The Data Flow Branch Predictor.

9.4 Hybrid Branch Predictors

Different branches in a program may be strongly correlated with different types of history. Because of this, some branches may be accurately predicted with global history-based predictors, while others are more strongly correlated with local history. Programs typically contain a mix of such branch types, and for example, choosing to implement a global history-based predictor may yield poor prediction accuracies for the branches that are more strongly correlated with their own local history.

To a certain degree, the alloyed branch predictors address this issue, but a tradeoff must be made between the number of global history bits used and the number of local history bits used. Furthermore, the alloyed branch predictors cannot effectively take advantage of predictors that use other forms of information, such as the loop predictor.

This section describes algorithms that employ two or more single-scheme branch prediction algorithms and combine these multiple predictions together to make one final prediction.

9.4.1 The Tournament Predictor

The simplest and earliest proposed multischeme branch predictor is the *tournament* algorithm [McFarling, 1993]. The predictor consists of two component predictors P_0 and P_1 and a *meta-predictor* M . The component predictors can be any of the single-scheme predictors described in Section 9.3, or even one of the hybrid predictors described in this section.

The meta-predictor M is a table of 2-bit counters indexed by the low-order bits of the branch address. This is identical to the lookup phase of Smith_2 , except that a (meta-)prediction of zero indicates that P_0 should be used, and a (meta-)prediction of one indicates that P_1 should be used (the meta-prediction is made from the most-significant bit of the counter). The meta-predictor makes a prediction of which predictor will be correct.

After the branch outcome is available, P_0 and P_1 are updated according to their respective update rules. Although the meta-predictor M is structurally identical to Smith_2 , the update rules (i.e., state transitions) are different. Recall that the 2-bit counters used in the predictors are finite state machines (FSMs), where the inputs are typically the branch outcome and the previous state of the FSM. For the meta-predictor M , the inputs are now c_0 , c_1 , and the previous FSM state, where c_i is one if P_i predicted correctly. Table 9.2 lists the state transitions. When P_1 's prediction was correct and P_0 mispredicted, the corresponding counter in M is incremented, saturating at a maximum value of 3. Conversely, when P_1 mispredicts and P_0 predicts correctly, the counter is decremented, saturating at zero. If both P_0 and P_1 are correct, or both mispredict, the counter in M is unmodified.

Figure 9.25a illustrates the hardware for the tournament selection mechanism with two generic component predictors P_0 and P_1 . The prediction lookups on P_0 , P_1 , and M are all performed in parallel. When all three predictions have been made, the meta-prediction is used to drive the select line of a multiplexer to choose between the predictions of P_0 and P_1 . Figure 9.25b illustrates an example tournament

Table 9.2
Tournament meta-predictor update rules

c_0 (P_0 Correct?)	c_1 (P_1 Correct?)	Modification to M
0	0	Do nothing
0	1	Saturating increment
1	0	Saturating decrement
1	1	Do nothing

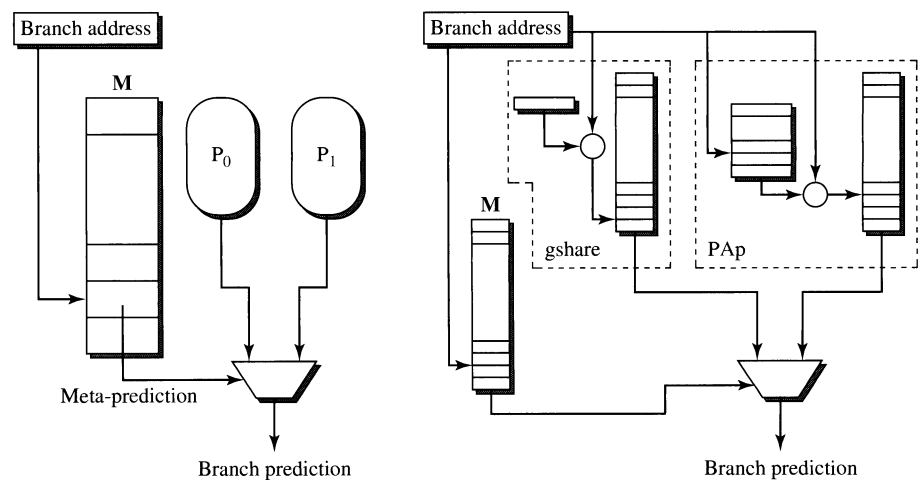


Figure 9.25
(a) The Tournament Selection Mechanism. (b) Tournament Hybrid with gshare and PAp.

selection predictor with gshare and PAp component predictors. A hybrid predictor similar to the one depicted in Figure 9.25b was implemented in the Compaq Alpha 21264 microprocessor [Kessler, 1999]. The local history component used a 1024-entry BHT with 10-bit per-branch histories. This 10-bit history is then used to index into a single 1024-entry PHT. The global history component uses a 12-bit history that indexes into a 4096-entry PHT of 2-bit counters. The meta-predictor also uses a 4096-entry table of counters.

Like the two-level branch predictors, the tournament's meta-predictor can also make use of branch history. It has been shown that a global branch outcome history hashed with the PC (similar to gshare) provides better overall prediction accuracies [Chang et al., 1995].

Either or both of the two components of a tournament hybrid predictor may themselves be hybrid predictors. By recursively arranging multiple tournament meta-predictors into a tree, any number of predictors may be combined [Evers, 2000].

9.4.2 Static Predictor Selection

Through profiling and program-based analysis, reasonable branch prediction rates can be achieved for many programs with static branch prediction. The downside of static branch prediction is that there is no way to adapt to unexpected branch behavior, thus leaving the possibility for undesirable worst-case behaviors. Grunwald et al. [1998] proposed using profiling techniques, but limited only to the meta-predictor. The entire multischeme branch predictor supports two or more component predictors, all which may be dynamic. The selection of which component to use is determined statically and encoded in the branch instruction as branch hints. The meta-predictor requires no additional hardware except for a single multiplexer to select between the component predictors' predictions.

The proposed process of determining the static meta-predictions is a lot more involved than traditional profiling techniques. Training sets are used to execute the programs to be profiled, but the programs are not executed on native hardware. Instead, a processor simulator is used to fully simulate the branch prediction structures in addition to the functional behavior of the program. The component predictor that is correct with the highest frequency is selected for each static branch. This may not be practical since the simulator may be very slow and full knowledge of the component branch predictors' implementations may not be available.

There are several advantages to the static selection mechanism. The first is that the hardware cost is negligible (a single additional n -to-1 multiplexer for n component predictors). The second advantage is that each static branch is assigned to one and only one component branch predictor. This means that the average number of static branches per component is reduced, which alleviates some of the problems of conflict and capacity aliasing. Although meta-predictions are performed statically, the underlying branch predictions still incorporate dynamic information, thus reducing the potential effects of worst-case branch patterns. The disadvantages include the overhead associated with simulating branch prediction structures during the profiling phase, the fact that the branch hints are not available until after the instruction fetch has been completed, and the fact that the number of component predictors is limited by the number of hint bits available in a single branch instruction.

9.4.3 Branch Classification

The branch classification meta-prediction algorithm is similar to the static selection algorithm and may even be viewed as a special case of static selection [Chang et al., 1994]. A profiling phase is first performed, but, in contrast to static selection, only the branch taken rates are collected (similar to the profile-based static branch prediction techniques described in Section 9.2.4). Each static branch is placed in one of six branch classes depending on its taken rate. Those which are heavily biased in one direction, defined as having a taken rate or not-taken rate of less than 5%, are statically predicted. The remaining branches are predicted using a tournament hybrid method.

The overall predictor has the structure of a static selection multischeme predictor with three components (P_0 , P_1 , and P_2). P_0 is a static not-taken branch predictor. P_1 is a static taken branch predictor. P_2 is itself another multischeme branch predictor, consisting of a tournament meta-predictor M and two component predictors, $P_{2,0}$ and $P_{2,1}$. The two component predictors of P_2 can be chosen to be any dynamic or static branch prediction algorithms, but are typically a global history predictor and a local history predictor. The branch classification algorithm has the advantage that easily predicted branches are removed from the dynamic branch prediction structures, thus reducing the number of potential sources for aliasing conflicts. This is similar to the benefits provided by branch filtering.

Figure 9.26 illustrates the hardware for a branch classification meta-predictor with static taken and non-taken predictors, as well as two unspecified generic components $P_{2,0}$ and $P_{2,1}$, and a tournament selection meta-predictor to choose between the two dynamic components. Similar to the static hybrid selection

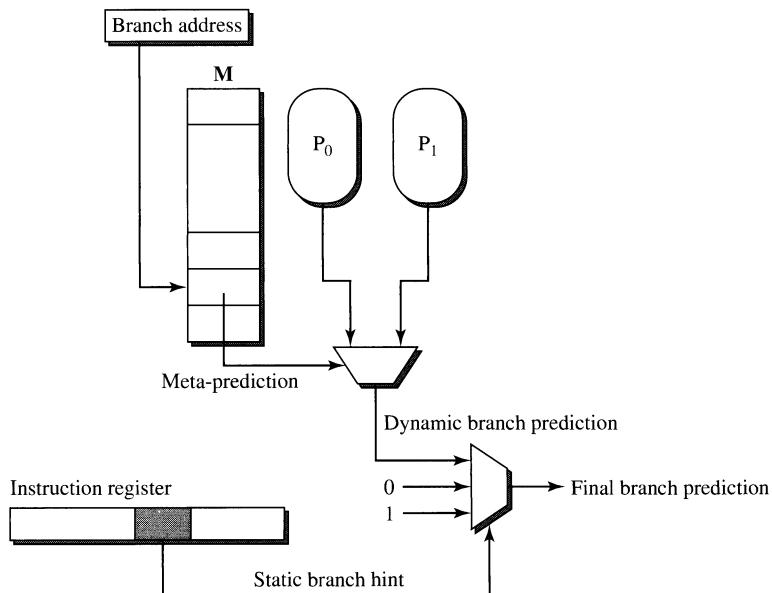


Figure 9.26

The Branch Classification Mechanism.

mechanism, the branch classification hint is not available to the predictor until after instruction fetch has completed.

9.4.4 The Multihybrid Predictor

Up to this point, none of the multischeme meta-predictors presented are capable of *dynamically* selecting from more than two component predictors (except for recursively using the tournament meta-predictor). By definition, a single tournament meta-predictor can only choose between two components. The static selection approach cannot dynamically choose any of its components. The branch classification algorithm can statically choose one of three components, but the dynamic selector used only chooses between two components.

The multihybrid branch predictor does allow the dynamic selection between an arbitrary number of component predictors [Evers et al., 1996]. The lower bits of the branch address are used to index into a table of prediction selection counters. Each entry in the table consists of n 2-bit saturating counters, $c_1, c_2, \dots, c_n$, where c_i is the counter corresponding to component predictor P_i . The components that have been predicting well have higher counter values. The meta-prediction is made by selecting the component whose counter value is 3 (the maximum) and a predetermined priority ordering is used to break ties. All counters are initialized to 3, and the update rules guarantee that at least one counter will have the value of 3. To update the counters, if at least one component with a counter value of 3 was correct, then the counter values corresponding to components that mispredicted are decremented (saturating at zero). Otherwise, the counters corresponding to components that predicted correctly are incremented (saturating at 3).

Figure 9.27 illustrates the hardware organization for the multihybrid meta-predictor with n component predictors. The branch address is used to look up an entry in the table of prediction selection counters, and each of the n counters is

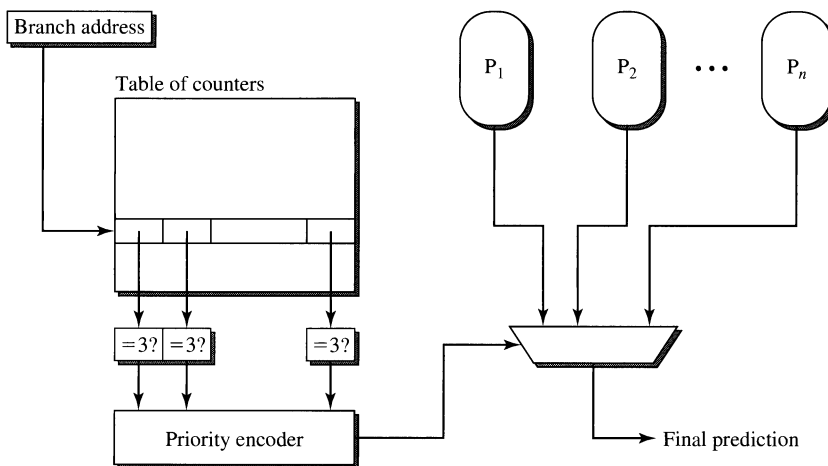


Figure 9.27

The Multihybrid Predictor.

checked for a value of 3. A priority encoder generates the index for the component with a counter value of 3 and the highest priority in the case of a tie. The index signal is then forwarded to the final multiplexer that selects the final prediction.

Unlike the static selection or even the branch classification meta-prediction algorithms, the multihybrid meta-predictor is capable of dynamically handling any number of component branch predictors.

9.4.5 Prediction Fusion

All the hybrid predictors described so far use a *selection* mechanism to choose one out of n predictions. By singling out a single predictor, selection-based hybrids throw out any useful information conveyed by the other predictors. Another approach, called *prediction fusion*, attempts to combine the predictions from all n individual predictors in making the final prediction [Loh and Henry, 2002]. This allows the hybrid predictor to leverage the information available from all component predictors, potentially making use of both global- and local-history components, or short- and long-history components, or some combination of these.

Prediction fusion covers a wide variety of predictors. Selection-based hybrid predictors are special cases of fusion predictors where the fusion mechanism ignores $n - 1$ of the inputs. The gskewed predictor can be thought of as a prediction fusion predictor that uses three gshare predictors with different hashing functions as inputs, and a majority function as the fusion mechanism.

One fusion-based hybrid predictor is the *fusion table*. Like the multihybrid predictor, the fusion table can take the predictions from an arbitrary number of subpredictors. For n predictors, the fusion table concatenates the corresponding n predictions together into an index. This index, combined with bits from the PC and possibly the global branch history form a final index into a table of saturating counters. The most significant bit of the indexed saturating counter provides the final prediction. This is illustrated in Figure 9.28.

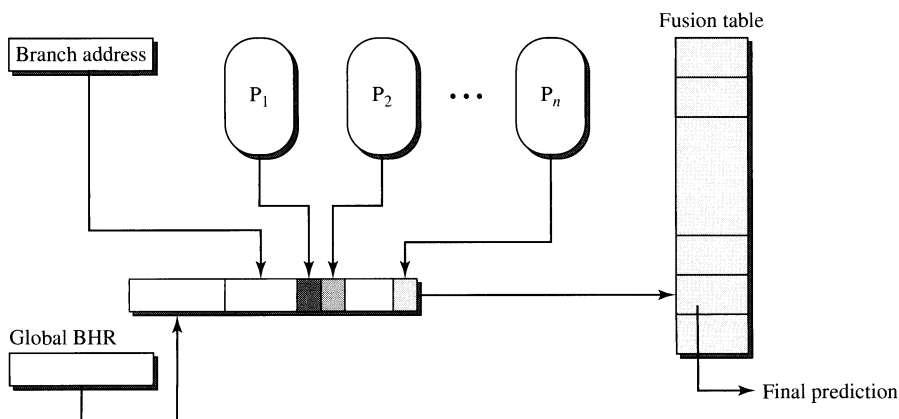


Figure 9.28

The Fusion Table Hybrid Predictor.

The fusion table provides a way to correlate branch outcomes to multiple branch predictions. The fusion table can remember any arbitrary mapping of predictions to branch outcome. For example, in the case where a branch is always taken if exactly one of two predictors is taken, the entries in the fusion table that correspond to this situation will be trained to predict taken, while the other entries that correspond to the predictor both predicting taken or both predicting not-taken will train to predict not-taken.

The fusion table hybrid predictor is very effective because it is very flexible. With a combination of global- and local-history components, and short- and long-history components, the fusion table can accurately capture a wide array of branch behaviors.

9.5 Other Instruction Flow Issues and Techniques

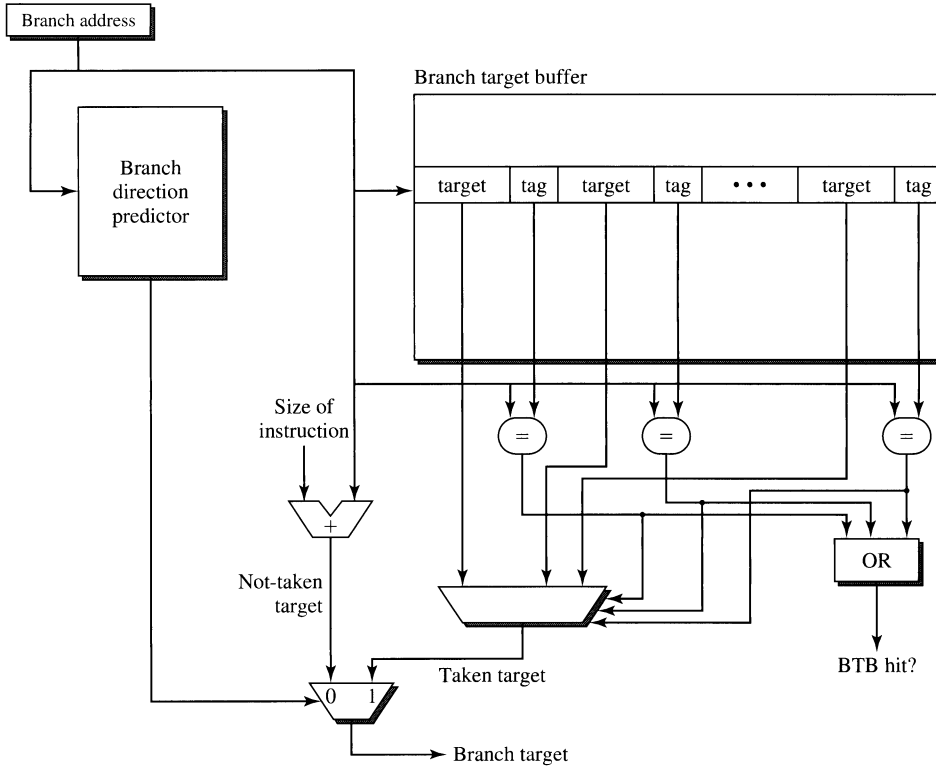
Predicting the direction of conditional branches is only one of several issues in providing a high rate of instruction fetch. This section covers these additional problems such as taken-branch target prediction, branch confidence prediction, predictor-cache organizations and interactions, fetching multiple instructions in parallel, and coping with faster clock speeds.

9.5.1 Target Prediction

For conditional branches, predicting whether the branch is taken or not-taken is only half of the problem. After the direction of a branch is known, the actual target address of the next instruction along the predicted path must also be determined. If the branch is predicted to be not-taken, then the target address is simply the current branch's address plus the size of an instruction word. If the branch is predicted to be taken, then the target will depend on the type of branch. Target prediction must also cover unconditional branches (branches that are always taken).

There are two common types of branch targets. Branch targets may be PC-relative, which means that the taken target is always at the current branch's address plus a constant (the constant may be negative). A branch target can also be *indirect*, which means that the target is computed at run time. An indirect branch target is read from a register, sometimes with a constant offset added to the contents of the register. Indirect branches are frequently used in object-oriented programs (such as the C++ vtable that determines the correct method to invoke for classes using inheritance), dynamically linked libraries, subroutine returns, and sometimes multitarget control constructs (i.e., C switch statements).

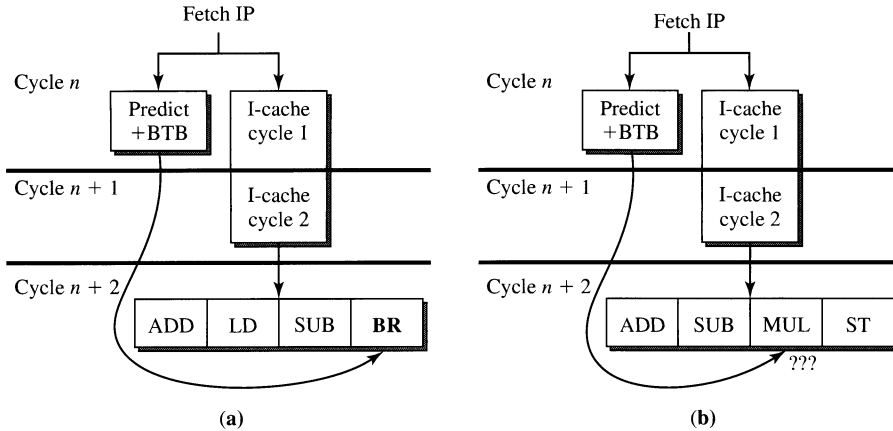
9.5.1.1 Branch Target Buffers. The target of a branch is usually predicted by a branch target buffer (BTB), sometimes also called a branch target address cache (BTAC) [Lee and Smith, 1984]. The BTB is a cache-like structure that stores the last seen target address for a branch instruction. When making a branch prediction, the traditional branch predictor provides a predicted direction. In parallel, the processor uses the current branch's PC to index into the BTB. The BTB is typically a tagged structure, often implemented with some degree of set associativity.

**Figure 9.29**

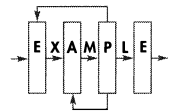
The Branch Target Buffer, a Generic Branch Predictor, and the Target Selection Logic.

Figure 9.29 shows the organization of the branch predictor and the BTB. If the branch predictor predicts not-taken, the target is simply the next sequential instruction. If the branch predictor predicts taken and there is a hit in the BTB, then the BTB's prediction is used as the next instruction's address. It is also possible that there is a taken-branch prediction, but there is a miss in the BTB. In this situation, the processor may stall fetching until the target is known. If the branch has a PC-relative target, then the fetch only stalls for a few cycles to wait for the completion of the instruction fetch from the instruction cache, the target offset extraction from the instruction word, and the addition of the offset to the current PC to generate the actual target. Another approach is to fall back to the not-taken target on a BTB miss.

Different strategies may be used for maintaining the information stored in the BTB. A simple approach is to store the targets of all branches encountered. A slightly better use of the BTB is to only store the targets of taken branches. This is because if a branch is predicted to be not taken, the next address is easily computed. By filtering out the not-taken targets, the prediction rate of the BTB may be improved by a decrease in interference.

**Figure 9.30**

Timing Diagram Illustrating That a Branch Prediction Occurs before the Instruction Fetch Completes: (a) When a Branch Is Present in the Fetch Block, and (b) When There Is No Branch Present in the Fetch Block.



In a pipelined processor, the instruction cache access may require multiple cycles to fetch an instruction. After a branch target has been predicted, the processor can immediately proceed with the fetch of the next instruction. There is a potential problem in that until the instruction has been fetched and decoded, how does the processor know if the next instruction is a branch or not? Figure 9.30 illustrates a branch predictor with a two-cycle instruction cache. In cycle n , the current branch address (IP) is fed to the branch predictor and BTB to predict the target of the next fetch block. At the same time, the instruction cache access for the current block is started. By the end of cycle n , the branch predictor and BTB have provided a direction and target prediction for the branch highlighted in bold in Figure 9.30(a).

Note that during cycle n when the branch prediction is made, it is not known that there will be a branch in the corresponding block. Figure 9.30(b) shows an example where there are no branches present in the fetch block. Since there are no branches, the next block to fetch is simply the next sequential block. But during cycle n when the branch prediction is made, the predictor does not know that there are no branches, and may even provide a target address that corresponds to a taken branch! A predicted taken branch that has no corresponding branch instruction is sometimes called a *phantom* branch or a *bogus* branch. In cycle $n+2$, the decode logic can detect that there are no branches present and, if there was a taken-branch prediction, the predictor and instruction cache accesses can be redirected to the correct next-block address. Phantom branches incur a slight performance penalty because the delay between branch prediction and phantom branch detection causes bubbles in the fetch pipeline.

When there are no branches present in a fetch block, the correct next-fetch address is the next sequential instruction block. This is equivalent to a not-taken

branch prediction, which is why only a *taken* branch prediction without a corresponding branch introduces a phantom branch. If the BTB is only ever updated with the targets of taken branches, and the next block of instructions does not contain *any* branches, then there will always be a BTB miss. If the processor uses a fallback to the not-taken strategy, then this will result in correct next-instruction address prediction when no branches are present, thus removing the phantom branches.

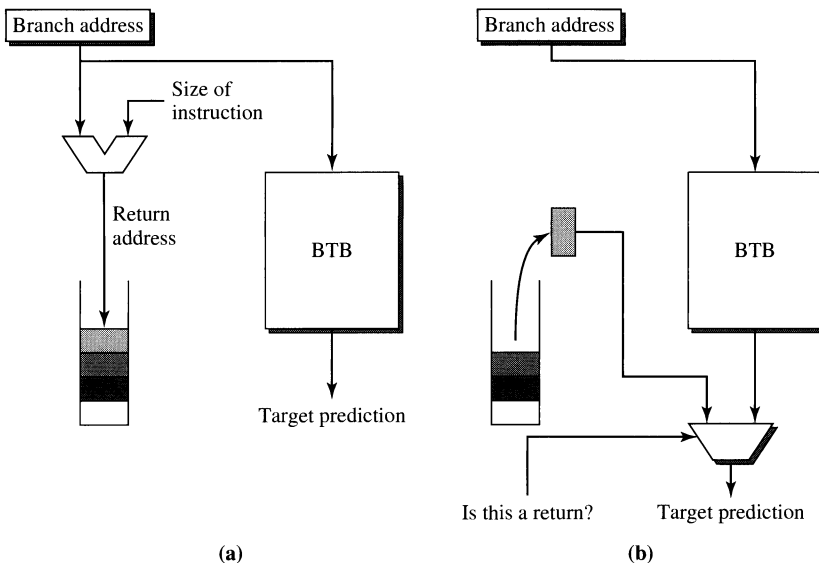
Address tags are typically fairly large, and so BTBs often use *partial resolution* [Fagin and Russell, 1995]. With partial resolution, only a subset of the tags are stored in the BTB entry. This allows for a decrease in the storage requirements, but opens up the opportunity for false hits. Two instructions with different addresses may both hit in the same BTB entry because the subset of bits used in the tag are identical, but there are differences in the address bits somewhere else. A BTB typically has fewer entries than a direction predictor because it must store an entire target address per entry (typically over 30 bits per entry), whereas the direction predictor only stores a small 2-bit counter per entry. The slight increase in mispredictions due to false hits is usually worth the decrease in structure size provided by partial resolution. Note that false hits can enable phantom branches to occur again, but if the false hit rate is low, then this will not be a serious problem.

9.5.1.2 Return Address Stack. Function calls frequently occur in programs. Both the jump into the function and the jump back out (the return) are usually unconditional branches. The target of a jump into a function is typically easy to predict. A branch instruction that jumps to `printf` will likely jump to the same place every time it is encountered. On the other hand, the return from the `printf` function may be difficult to predict because `printf` could be called from many different places in a program.

Most instruction set architectures support subroutine calls by providing a means of storing the subroutine return address. When executing a jump to a subroutine, the address of the instruction that sequentially follows the jump is stored into a register. This address is then typically stored on the stack and used as a jump address at the end of the function when the return is called.

The return address stack (RAS) is a special branch target predictor that only provides predictions for subroutine returns [Kaeli and Emma, 1991]. When a jump into a function happens, the return address is pushed onto the RAS, as shown in Figure 9.31(a). During this initial jump, the RAS does not provide a prediction and the target must be predicted from the regular BTB. At some later point in the program when the program returns from the subroutine, the top entry of the RAS is popped and provides the correct target prediction as shown in Figure 9.31(b). The stack can store multiple return addresses, and so returns from nested functions will also be properly predicted.

The return address stack does not guarantee perfect prediction of return target addresses. The stack has limited capacity, and therefore functions that are too deeply nested will cause a stack overflow. The RAS is often implemented as a circular buffer, and so an overflow will cause the most recent return address to overwrite the oldest return address. When the stack unwinds to the return that was

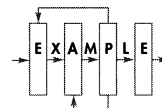
**Figure 9.31**

(a) Return Address Push on Jump to Subroutine. (b) Return Address Pop on Subroutine Return.

overwritten, a target misprediction will occur. Another source of RAS misprediction is irregular code that does not have matched subroutine calls and returns. Usage of the C library functions `setjmp` and `longjmp` could result in the RAS containing many incorrect targets.

Usage of the RAS requires knowing whether a branch is a function call or a return. This information is typically not available until after the instruction has been fetched. For a subroutine call, the target is predicted by the BTB, and so this will not introduce any bubbles into the fetch pipeline. For a subroutine return, the BTB may provide an initial target prediction. After the instruction has actually been fetched, it will be known that it is a return. At this point, the instruction flow may be corrected by squashing any instructions incorrectly fetched (or in the process of being fetched) and then resuming fetch from the return target provided by the RAS. Without the RAS, the target misprediction would not be detected until the return address has been loaded from the program stack into a register and the return instruction has been executed.

Return address stacks are implemented in almost all current microprocessors. An example is the Pentium 4, which uses a 16-entry return address stack [Hinton et al., 2001]. The RAS is also sometimes referred to as a *return stack buffer* (RSB).



9.5.2 Branch Confidence Prediction

Some branches are easy to predict, while others cause great trouble for the branch predictor. Branch confidence prediction does not make any attempt to predict the outcome of a branch, but instead makes a prediction about a branch prediction.

The purpose of branch confidence prediction is to guess or estimate how certain the processor is about a particular branch prediction. For example, the selective branch inversion technique (see Section 9.3.2.6) switches the direction of the initial branch prediction when the confidence is predicted to be very low. The confidence prediction detects cases where the branch direction predictor is consistently doing the wrong thing, and then selective branch inversion (SBI) uses this information to rectify the situation. There are many other applications of branch confidence information. This section first discusses techniques for predicting branch confidence, and then surveys some of the applications of branch confidence prediction.

9.5.2.1 Prediction Mechanisms. With branch confidence prediction, the information used is whether branch predictions are correct or not, as opposed to whether the prediction is taken or not-taken [Jacobson et al., 1996]. Figure 9.32 shows a branch confidence predictor that uses a global branch outcome history as context in a fashion similar to a gshare predictor, but the PHT has been replaced by an array of *correct/incorrect registers* (CIRs). A CIR is a shift register similar to a BHR in conventional branch predictors, but instead of storing the history of branch directions, the CIR stores the history of whether or not the branch was correctly predicted. Assuming that a 0 indicates a correct prediction, and a 1 indicates a misprediction, four correct predictions followed by two mispredictions followed by three more correct predictions would have a CIR pattern of 000011000.

To generate a final confidence prediction of high confidence or low confidence, the CIR must be processed by a *reduction function* to produce a single bit. The ones-counting approach counts the number of 1s in the CIR (that is, the number of mispredictions). The confidence predictor assumes that a large number of recent mispredictions indicates that future predictions will also likely be incorrect. Therefore, a higher ones-count indicates lower confidence. A more efficient implementation replaces the CIR shift register with a saturating counter. Each time

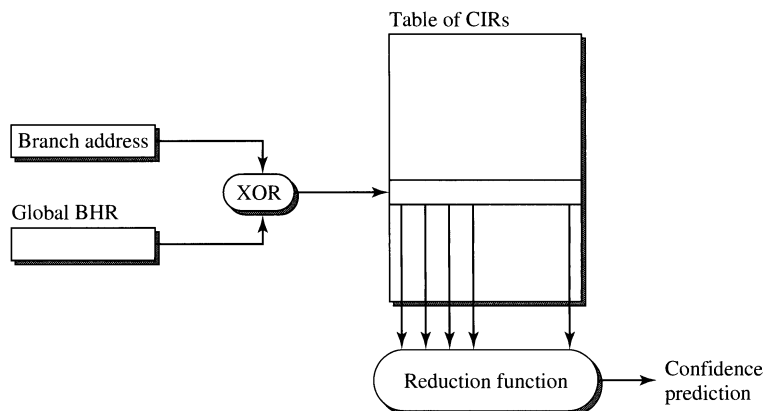


Figure 9.32
A Branch Confidence Predictor.

there is a correct prediction, the counter is incremented. The counter is decremented for a misprediction. If the counter has a large value, then it means that the branch predictions have been mostly correct, and therefore a large CIR counter value indicates high confidence. To detect n consecutive correct predictions, a shift register CIR needs to be n bits wide. On the other hand, a counter-based CIR only requires $\lceil \log_2 n \rceil$ bits. An alternative implementation uses resetting counters where each misprediction causes the counter to reset to zero instead of decrementing the counter. The counter value is now equal to the number of branches since the last misprediction seen by the CIR. Because the underlying branch prediction algorithms are already very accurate, the patterns observed in the shift-based CIRs are dominated by all zeros (no recent mispredictions) or a single one (only one recent misprediction). Since the resetting counter tracks the distance since the last misprediction, it approximately represents the same information.

The structure of the branch confidence predictor is very similar to branch direction predictors. Some of the more advanced techniques used for branch direction predictors could also be applied to confidence predictors.

9.5.2.2 Applications. Besides the already discussed selective branch inversion technique, branch confidence prediction has many other potential applications.

An alternative approach to predicting conditional branches and speculatively going down one path or the other is to fetch and execute from both the taken and not-taken paths at the same time. This technique, called *eager execution*, guarantees that the processor will perform some useful work, but it also guarantees that some of the instructions fetched and executed will be discarded. Allowing the processor to “fork” every conditional branch into two paths rapidly becomes very expensive since the processor must be able to track all the different paths and flush out different sets of instructions as different branches are resolved. Furthermore, performing eager execution for a highly predictable branch wastes resources. The wrong path will use up fetch bandwidth, issue slots, functional units, and cache bandwidth that could have otherwise been used for the correct path instructions. *Selective eager execution* limits the harmful effects of uncontrolled eager execution by limiting dual-path execution to only those branches that are deemed to be difficult, i.e., low-confidence branches [Klauser et al., 1998]. A variation of eager execution called *disjoint eager execution* is discussed in Chapter 11 [Uht, 1997].

Branch mispredictions are a big reason for performance degradations, but they also represent a large source of wasted power and energy. All the instructions on a mispredicted path are eventually discarded, and so all the power spent on fetching, scheduling, and executing these instructions is energy spent for nothing. Branch confidence can be used to decrease the power consumption of the processor. When a low-confidence branch is encountered, instead of making a branch prediction that has a poor chance of being correct, the processor can simply stall the front-end and wait until the actual branch outcome has been computed. This reduces instruction-level parallelism by covering up any parallelism blocked by this control dependency, but greatly reduces the power wasted by branch mispredictions.

Branch confidence can also be used to modify the fetch policies of simultaneously multithreaded (SMT) processors (see Chapter 11). An SMT processor fetches and executes instructions from multiple threads using the same hardware. The fetch engine in an SMT processor will fetch instructions for one thread in a cycle, and then depending on its fetch policy, it may choose to fetch instructions from another thread on the following cycle. If the current thread encounters a low-confidence branch, then the fetch engine could stop fetching branches from the current thread and start fetching instructions from another thread that are more likely to be useful. In this fashion, the execution resources that would have been wasted on a likely branch misprediction are usefully employed by another thread.

9.5.3 High-Bandwidth Fetch Mechanisms

For a superscalar processor to execute multiple instructions per cycle, the fetch engine must also be able to fetch multiple instructions per cycle. Fetching instructions from an instruction cache is typically limited to accessing a single cache line per cycle. Taken branches disrupt the instruction delivery stream from the instruction cache because the next instruction to fetch is likely to be in a different cache line. For example, consider a cache line that stores four instruction words where one is a taken branch. If the taken branch is the first instruction in the cache line, then the instruction cache will only provide one useful instruction (i.e., the branch). If the taken branch is in the last position, then the instruction cache can provide four useful instructions.

The maximum number of instructions fetched per cycle is bounded by the number of words in the instruction cache line (assuming a limit of one instruction cache access per cycle). Unfortunately, increasing the size of the cache line has only very limited effectiveness in increasing the fetch bandwidth. For typical integer applications, the number of instructions between the target of a branch and the next branch (i.e., a basic block) is only about five to six instructions. Assuming six instructions per basic block, and that 60% of branches are taken, the expected number of instructions between taken branches is 15. Unfortunately, there are many situations where the rate of taken branches is close to 100% (for example, a loop). In these cases, the instruction fetch engine will only be able to provide at most one iteration of the loop per cycle.

A related problem is the situation where a block of instructions is split across cache lines. If the first of four sequential instructions to be fetched is located at the end of a cache line, then two cache accesses are necessary to fetch all four instructions.

This section describes two mechanisms for providing instruction fetch bandwidth that can handle multiple basic blocks per cycle.

9.5.3.1 The Collapsing Buffer. The *collapsing buffer* scheme uses a combination of an interleaved BTB to provide multiple target predictions, a banked instruction cache to provide more than one line of instructions in parallel, and masking and alignment (collapsing) circuitry to compact the statically nonsequential instructions [Conte et al., 1995].

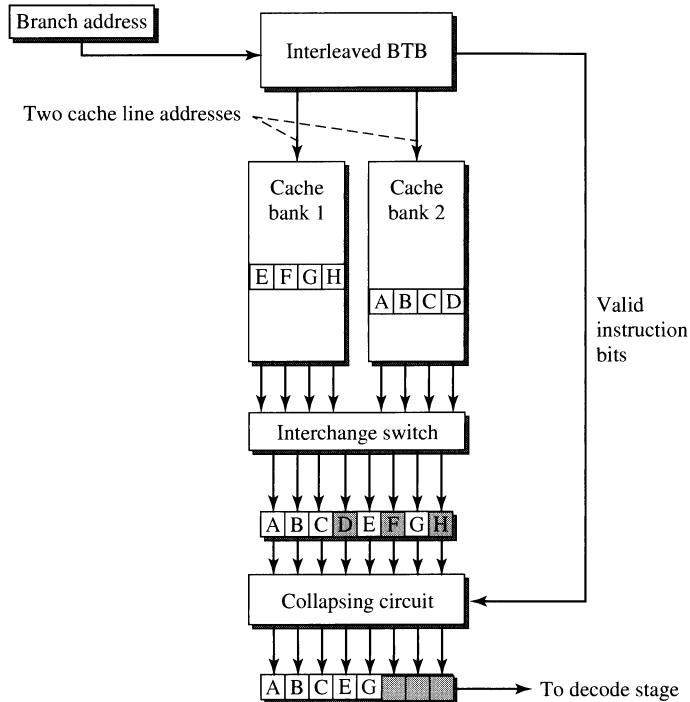


Figure 9.33

The Collapsing Buffer Fetch Organization.

Figure 9.33 shows the organization of the collapsing buffer. In this figure, the cache lines are four instruction words wide, and the cache has been broken into two banks. The instructions to be fetched are A, B, C, E, and G. In a conventional instruction cache, only instructions A, B, and C would be fetched in a single cycle. The branch target buffer provides the predictions that instruction C is a taken branch (with target address E) and that instruction E is also a taken branch with target address G. One cache bank fetches the cache line containing instructions A, B, C, and D, and the other cache bank provides instructions E, F, G, and H. If both cache lines are in the same bank, then the collapsing buffer fetch mechanism will only provide at most one cache line's worth of instructions.

After the two cache lines have been fetched, the cache lines go through an interchange switch that swaps the two lines if the second cache line contains the earlier instructions. The interleaved BTB provides valid instruction bits that specify which entries in the cache lines are part of the predicted path. A collapsing circuit, implemented with shifting logic, collapses the disparate instructions into one contiguous sequence to be forwarded to the decoder. The instructions to be removed are shaded in Figure 9.33. Note that in this example, the branch C crosses cache lines. The branch E is an *intra*block branch where the branch target resides in the

same cache line as the branch. In this example, the collapsing buffer has provided five instructions whereas a traditional instruction cache would only fetch three.

A shift-logic based collapsing circuit suffers from a long latency. Another way to implement the collapsing buffer is with a crossbar. A crossbar allows an arbitrary permutation of its inputs, and so even the interchange network is not needed. Because of this, the overall latency for interchange and collapse may be reduced, despite the relatively complex nature of crossbar networks.

The collapsing buffer adds some complex circuitry to the fetch path. The interchange switch and collapsing circuit add considerable latency to the front-end pipeline. This extra latency would take the form of additional pipeline stages, which increases the branch misprediction penalty. The organization of the collapsing buffer is difficult to scale to support fetching from more than two cache lines per cycle.

9.5.3.2 Trace Cache. The collapsing buffer fetch mechanism highlights the fact that many dynamically sequential instructions are not physically located in contiguous locations. Taken branches and cache line alignment problems frequently disrupt the fetch engine’s attempt to provide a continuous high-bandwidth stream of instructions. The *trace cache* attempts to alleviate this problem by storing logically sequential instructions in the same consecutive physical locations [Friendly et al., 1997; Rotenberg et al., 1996; 1997].

A *trace* is a dynamic sequence of instructions. Figure 9.34(a) shows a sequence of instructions to be fetched and their locations in a conventional instruction cache. Instruction B is a predicted taken branch to C. Instructions C and D are split across two separate cache lines. Instruction D is another predicted taken branch to E. Instructions E through J are split across two cache lines, but this is to be expected since there are more instructions in this group than the width of a cache line. With a conventional fetch architecture, it will take at least five cycles to fetch these 10 instructions because the instructions are scattered over five different

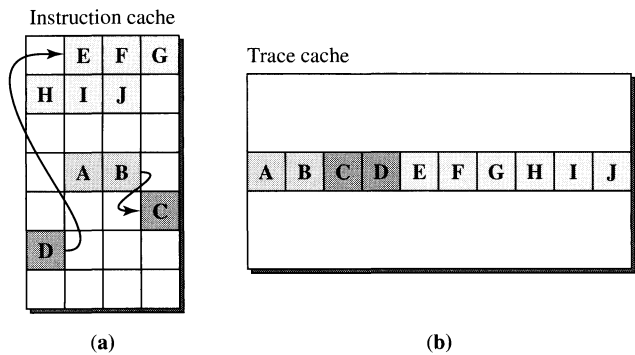
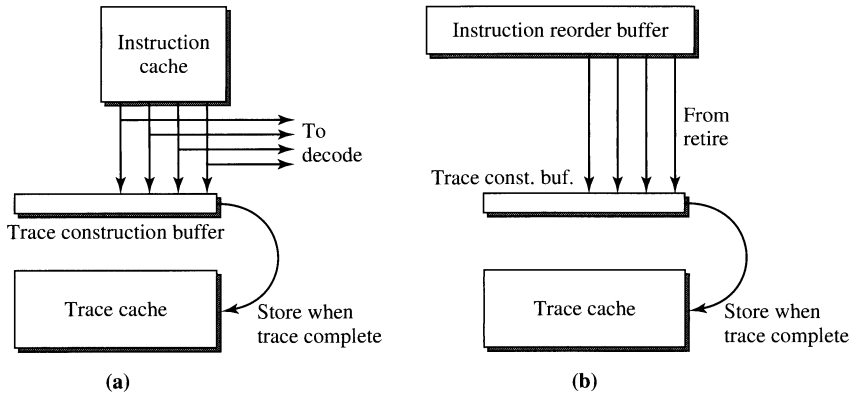


Figure 9.34
(a) Ten Instructions in an Instruction Cache. (b) The Same 10 Instructions in a Trace Cache.

**Figure 9.35**

(a) Fetch-Time Trace Construction. (b) Completion-Time Trace Construction.

cache lines. Even with a collapsing buffer, it would still take three cycles (maximum fetch rate of two lines per cycle).

The trace cache takes a different approach. Instead of attempting to fetch from multiple locations and stitch the instructions back together like the collapsing buffer, the trace cache stores the entire trace in one physically contiguous location as shown in Figure 9.34(b). The trace cache can deliver this entire 10-instruction trace in a single lookup without any complicated reshuffling or realignment of instructions.

Central to the trace cache fetch mechanism is the task of trace construction. Trace construction primarily occurs at one of two locations. The first possibility is to perform trace construction at fetch time, as shown in Figure 9.35(a). As instructions are fetched from the conventional instruction cache, a trace construction buffer stores the dynamic sequence of instructions. When the trace is complete, which may be determined by various constraints such as the width of the trace cache or a limit on the number of branches per trace, this newly constructed trace is stored into the trace cache. In the future, when this same path is encountered, the trace cache can provide all of the instructions in a single access.

The other point for trace construction is at the back end of the processor when instructions retire. Figure 9.35(b) shows how as the processor back end retires instructions in order, these instructions are placed into a trace construction buffer. When the trace is complete, the trace is stored into the trace cache and a new trace is started. One advantage of back-end trace construction is that the circuitry is not in the branch misprediction pipeline, and the trace constructor may take more cycles to construct traces.

A trace entry consists of the instructions in the trace, and the entry also contains tags corresponding to the starting points of each basic block included in the trace. To perform a lookup in the trace cache, the fetch engine must provide the trace cache with the addresses of all basic block starting addresses on the predicted path. If all addresses match, then there is a trace cache hit. If some prefix of the

addresses match (e.g., the first two addresses match but the third does not), then it is possible to provide only the subset of the trace that corresponds to the predicted path. For a high rate of fetch, the trace cache requires the front end to perform multiple branch predictions per cycle. Adapting conventional branch predictors to perform multiple predictions while maintaining reasonable access latencies is a challenging design task.

An alternative to making multiple branch predictions per cycle is to treat a trace as the fundamental basic unit and perform *trace prediction*. Each trace has a unique identifier defined by the starting PC and the outcomes of all conditional branches in the trace. The trace predictor's output is one of these trace identifiers. This approach provides trace-level sequencing of instructions.

Even with trace-level sequencing, some level of instruction-level sequencing (i.e., conventional fetch) must still be provided. At the start of a program, or when a program enters new regions of code, the trace cache will not have constructed the appropriate traces and the trace predictor has not learned the trace-to-trace transitions. In this situation, a conventional instruction cache and branch predictor provide the instructions at a slower rate until the new traces have been constructed and the trace predictor has been properly trained.

The Intel Pentium 4 microarchitecture employs a trace cache, but no first-level instruction cache [Hinton et al., 2001]. Figure 9.36 shows the block-level organization of the Pentium 4 fetch and decode engine. When the trace cache is in use, the trace cache BTB provides the fetch addresses and next-trace predictions. If the predicted trace is not in the trace cache, then instruction-level sequencing occurs, but the instructions are fetched from the level-2 instruction/data cache. This increases the number of cycles to fetch an instruction when there is a trace cache miss. These instructions are then decoded, and these *decoded instructions* are stored in the trace cache. Storing the decoded instructions allows instructions fetched from the trace cache to skip over the decode stage of the pipeline.

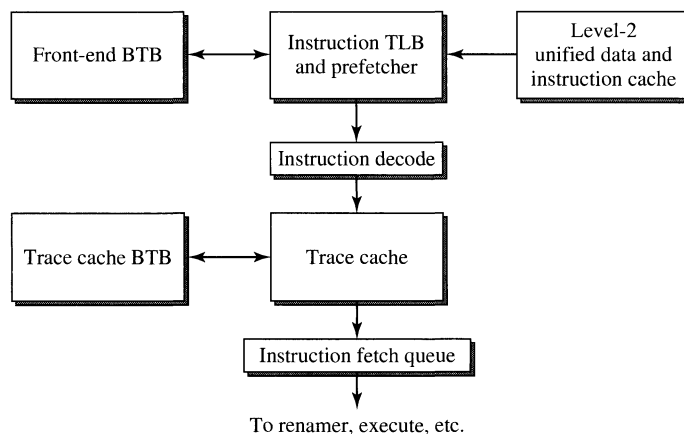


Figure 9.36

Intel Pentium 4 Trace Cache Organization.